

Algorithm Specification and Supporting Documentation

WEAVER: A Lattice Based Key Encapsulation Mechanism

Long Chen¹, Zidi Zhuang¹, Tanping Zhou², Jianhua Hou³, Si Gao¹, Yuyang Xiao¹,
Yingjie Hao¹, Shuo Chen², and Ruiqi Mi¹

¹Institute of Software, Chinese Academy of Sciences, China

²Engineering University of PAP, China

³Information Engineering University, PLA Cyberspace Force, China

May 8, 2026

Abstract

This document provides the complete specification, security analysis, and performance evaluation for [Algorithm Name], submitted to the New Generation Commercial Cryptography (NGCC) algorithm call by ICCS. [Algorithm Name] is a lattice-based Key Encapsulation (KE) protocol designed to achieve high efficiency and compact transmission sizes .

Contents

1	Introduction	3
1.1	Main Features and Advantages	3
2	Algorithm Description	5
2.1	Preliminaries and Notation	5
2.2	Data Encoding and Serialization	7
2.3	Algorithm Specification	8
2.3.1	Specification of IND-CPA-secure PKE	8
2.3.2	Specification of IND-CCA-secure KEM	13
2.4	Parameter Sets	14
3	Design Rationale	15
3.1	Underlying Hard Problems	15
3.2	Message Encoding Strategy and Error Correction	17
3.3	Parameter Selection Strategy	17
3.3.1	Algebraic Parameters (n, q)	17
3.3.2	Security Scaling Parameters (k, η_1, η_2)	18
3.3.3	Bandwidth Compression Parameters (d_t, d_u, d_v)	18
3.3.4	Error Correction Parameters (BCH and D_4 Lattice)	18
3.4	Decryption Failure Analysis	18
3.4.1	Layered Decoding Thresholds	19
3.4.2	Single Bit Failure Rate	19
3.4.3	Independence Assumption and Overall Decryption Failure Rate	20
3.5	Symmetric Primitives	21
3.5.1	Implicit Rejection FO Transform	21

4	Security Statement and Analysis	21
4.1	Theoretical Security	21
4.1.1	Hardness Assumption	22
4.1.2	From IND-CPA PKE to IND-CCA KEM	24
4.2	Concrete Security Estimates	25
4.3	Security Margin against Failure Boosting	27
4.3.1	Characterizing the Decryption Noise	27
4.3.2	Searchable Randomness under Q Queries	28
4.3.3	Conditional Noise Estimate and Failure Probability	28
4.3.4	Attack-complexity interpretation.	29
4.4	Empirical Validation of Error Independence	30
4.4.1	Methodology and Metrics	30
4.4.2	Experimental Results and Structural Underdispersion	31
4.4.3	Conclusion on Security Margin	31
4.5	Side-Channel Resistance and Implementation Security	31
4.6	Other Security Properties	31
5	Performance Evaluation and Implementation Analysis	31
5.1	Evaluation Methodology and Platforms	31
5.2	Software Performance on x86 Processors	32
5.2.1	Benchmarking Results (Intel Core i7)	32
5.3	Performance on Embedded Systems (ARM Cortex-M4)	32
5.3.1	Benchmarking Results (STM32F4)	32
5.4	Optimization Techniques	32
5.4.1	Vectorization with AVX2 (x86)	33
5.4.2	Optimizations for 32-bit Embedded Systems	33
5.5	Transmission and Storage Overhead	33
6	Features Statement	34
6.1	Innovation	34
6.2	Compatibility and Real-World Applicability	34
6.3	Flexibility and Scalability	34
6.4	Innovation	34
6.5	Compatibility and Real-World Applicability	34
A	Intellectual Property Statement	35
B	Test Vectors	35

1 Introduction

This document provides the complete specification, security analysis, and performance evaluation for WEAVER, a lattice-based Key Encapsulation Mechanism (KEM) submitted to the New Generation Commercial Cryptography (NGCC) algorithm call. WEAVER is designed to achieve an optimal balance between high computational efficiency, extremely compact transmission sizes, and robust post-quantum security.

At its core, WEAVER builds upon the Module Learning With Rounding (MLWR) framework but introduces a paradigm shift in how ciphertext compression and decryption failures are handled. We adopt a “Dancing with Imperfection” design philosophy: by completely eliminating explicitly sampled error vectors and aggressively compressing the transmitted elements, we naturally incur a high level of rounding noise. Instead of conservatively bounding this noise to avoid decryption failures, we leverage a highly efficient hierarchical error correction mechanism to safely absorb it. This pure rounding-based approach allows WEAVER to achieve highly competitive bandwidth profiles perfectly suited for real-world network protocols (such as IKEv2/IPsec) without sacrificing the rigorous security guarantees required by NGCC.

1.1 Main Features and Advantages

The distinct advantages of WEAVER stem from three primary theoretical and engineering innovations.

Hierarchical BCH Codes with Soft-Decision Repetition. To maintain a cryptographically negligible Decryption Failure Rate (DFR) under aggressive parameter compression, WEAVER departs from the traditional scalar rounding approach. We introduce a novel multi-level message encoding strategy. The shared secret μ is split into two parts: the higher bits $\bar{\mu}$ are protected by a standard Bose-Chaudhuri-Hocquenghem (BCH) code, while the highly noisy lower bits $\dot{\mu}$ are protected by a 4-way repetition code combined with soft-decision decoding. Specifically, each lower-significance message bit is spread across four distant polynomial coefficients at indices $i, i+64, i+128, i+192$. During decryption, instead of making hard decisions on individual coefficients, the receiver aggregates the absolute distances (L_1 norm) across all four dimensions before thresholding. This multi-dimensional soft-decision approach provides a substantial coding gain against the heavy rounding noise, and is further concatenated with a secondary BCH code to eliminate residual errors. This hierarchical design allows WEAVER to extract reliable bits from heavily distorted coefficients that would be irrecoverable by any standard rounding-based decoder.

Pure Rounding-Based Hardness and Security Amplification. In standard lattice-based schemes such as ML-KEM (Kyber), ciphertext compression is primarily treated as an engineering technique to reduce bandwidth, while cryptographic hardness relies on explicitly sampled Gaussian or binomial error vectors. WEAVER fundamentally changes this perspective by adopting a pure Learning With Rounding (LWR) design. We completely eliminate the explicit error vectors (e, e', e'') . Instead, the cryptographic noise naturally and entirely arises from our aggressive compression operations (e.g., retaining only $d_v = 4$ bits for the ciphertext component v). By analyzing the scheme through the lens of the LWR problem, the effective noise variance faced by an attacker is purely determined by the compression-induced rounding:

$$\sigma_{\text{eff}}^2 \approx \frac{q^2}{12 \cdot 2^{2d}}.$$

This inherent rounding noise is intentionally maximized to provide the necessary Core-SVP hardness against lattice reduction attacks (such as primal and dual attacks). This pure rounding-based approach achieves a dual benefit: it minimizes bandwidth by dropping as many bits as

possible, while simultaneously generating the cryptographic noise required for security, eliminating the computational overhead and randomness requirements of sampling explicit error vectors.

Prime Modulus Compatibility for NTT and LWR via Randomized Lifting. Historically, LWR-based schemes (e.g., Saber) have relied on power-of-two moduli to facilitate cost-free modular reduction and rounding, but this algebraic structure precludes the use of the highly efficient Number Theoretic Transform (NTT) for polynomial multiplication. Conversely, schemes utilizing NTT-friendly prime moduli (e.g., Kyber with $q = 3329$) benefit from fast arithmetic but do not naturally align with pure LWR-style hardness assumptions, since the deterministic decompression of public keys breaks the exact uniformity required for standard security reductions. WEAVER successfully bridges this divide. We retain the NTT-friendly prime modulus $q = 3329$ to ensure peak polynomial arithmetic performance. To rigorously justify the aggressive LWR-style compression over a prime field without resorting to non-standard assumptions, we introduce a novel **randomized lifting** mechanism (Inv_q) during encryption. By deterministically sampling a uniformly random preimage from the compression bucket, this mechanism perfectly restores the exact uniformity of the public key from the ciphertext’s perspective. This elegant technique provides a sound and tight provable-security foundation, allowing us to reduce the IND-CPA security of WEAVER directly to the standard decisional Module-LWR (MLWR) assumption, seamlessly combining the computational speed of NTT with the bandwidth efficiency of LWR.

2 Algorithm Description

2.1 Preliminaries and Notation

Modular reductions. For an even (resp. odd) positive integer α , we define $r_0 = r \bmod^\pm \alpha$ to be the unique element r_0 in the range $-\frac{\alpha}{2} < r_0 \leq \frac{\alpha}{2}$ (resp. $-\frac{\alpha-1}{2} \leq r_0 \leq \frac{\alpha-1}{2}$) such that $r_0 = r \bmod \alpha$. For any positive integer α , we define $r_0 = r \bmod^+ \alpha$ to be the unique element r_0 in the range $0 \leq r_0 < \alpha$ such that $r_0 = r \bmod \alpha$. When the exact representation is not important, we simply write $r \bmod \alpha$.

Polynomial Rings. Let $\mathbb{Z}$ denote the ring of integers. For a positive integer q , let $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. We denote its **least nonnegative residues** as $\mathbb{Z}_q^+ := \{0, 1, \dots, q-1\}$. Let n be a power of 2. In our concrete parameter sets, we use $n = 256$ for the lower security levels and $n = 512$ for the highest security level. We define $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$. Elements in the ring, $a \in \mathcal{R}_q$ are polynomials of degree less than n , which can be written as: $a = \sum_{i=0}^{n-1} a_i X^i, a_i \in \mathbb{Z}_q$, or equivalently denoted by its coefficient vector $a = (a_0, a_1, \dots, a_{n-1})$. When we want to uniquely decide its representation or emphasize its residue representatives, for example, we use $\mathcal{R}_{q^+}$ to denote the representation $a = (a_0, a_1, \dots, a_{n-1}), a_i \in \mathbb{Z}_q^+$.

Vectors and Matrices. We use bold lower-case letters (e.g., $\mathbf{u}, \mathbf{v}, \mathbf{s}$) for column vectors over $\mathcal{R}_q$, and bold upper-case letters (e.g., $\mathbf{A}$) for matrices over $\mathcal{R}_q$. For $\mathbf{u}, \mathbf{v} \in \mathcal{R}_q^k$, their inner product is

$$\mathbf{u}^T \mathbf{v} = \sum_{i=0}^{k-1} u_i v_i \in \mathcal{R}_q.$$

The transpose of a vector or matrix is denoted by $\mathbf{v}^T$ or $\mathbf{A}^T$. In the NTT domain, $\mathbf{u} \circ \mathbf{v}$ denotes coefficient-wise multiplication.

Distributions and Sampling. We write $x \leftarrow \$ S$ for uniform sampling from a finite set S . Let CBD_η denote the centered binomial distribution with parameter η . A sample from CBD_η is computed as

$$\sum_{i=1}^{\eta} (a_i - b_i),$$

where each $a_i, b_i \in \{0, 1\}$ is sampled uniformly and independently. We write $x \leftarrow \text{CBD}_\eta$ to denote sampling from this distribution. When sampling a polynomial in $\mathcal{R}_q$, all coefficients are sampled independently from CBD_η .

Compression and Canonical Decompression. For $x \in \mathbb{R}$, $\lceil x \rceil$ denotes rounding to the nearest integer, with ties broken upward, and $\lfloor x \rfloor$ denotes the floor function. These operations are extended coefficient-wise to polynomials and vectors.

For $x \in \mathbb{Z}_q$ and a compression parameter $d < \lceil \log_2 q \rceil$, define

$$\text{Compress}_q(x, d) := \left\lceil \frac{2^d}{q} \cdot (x \bmod^+ q) \right\rceil \bmod^+ 2^d,$$

and, for $y \in \mathbb{Z}_{2^d}$,

$$\text{Decompress}_q(y, d) := \left\lfloor \frac{q}{2^d} \cdot (y \bmod^+ 2^d) \right\rfloor \bmod^+ q.$$

These maps extend coefficient-wise to polynomials and vectors.

Randomized Lifting. We apply “Randomized Lifting” technique for public key’s decompression, in order to achieve strict security reduction. Namely, for each $y \in \mathbb{Z}_{2^d}^+$, define its preimage set:

$$\text{Preim}_q(y, d) := \{x \in \mathbb{Z}_q^+ : \text{Compress}_q(x, d) = y\}.$$

The randomized lifting map $\text{Inv}_q(\cdot, d) : y \mapsto x'$ is defined by:

$$\text{Inv}_q(y, d) := x' \bmod^+ q, \quad x' \leftarrow \$ \text{Preim}_q(y, d).$$

That is, $\text{Inv}_q(y, d)$ outputs a uniformly random element from the set of all residues in $\mathbb{Z}_q^+$ that compress to y . For polynomials and vectors, $\text{Inv}_q(\cdot, d)$ is applied coefficient-wise and independently across coefficients unless stated otherwise.

An alternative (equivalent) approach to defining $\text{Inv}_q(\cdot, d)$ is to first define “preimage error set” as:

$$\text{Preim}_q^\pm(y, d) = \{\text{Decompress}_q(y, d) - x \bmod^\pm q, \ x \in \mathbb{Z}_q^+ : \text{Compress}_q(x, d) = y\}.$$

then define $\text{Inv}_q(\cdot, d)$ as:

$$\text{Inv}_q(y, d) := \text{Decompress}_q(y, d) + e, \quad e \leftarrow \$ \text{Preim}_q^\pm(y, d)$$

Now, instead of using the canonical decompression for pk recover in `WEAVER.CPAPKE.Enc`, which was $t' = \text{Decompress}_q(\text{Compress}_q(t, d), d)$, we apply the randomized lift:

$$t_{\text{lift}} \leftarrow \text{Inv}_q(\text{Compress}_q(t, d), d).$$

The Property Achieved by Randomized Lifting. By incorporating $\text{Inv}_q(\cdot, d)$, the public key after decompression in the encryption function recovers to be uniformly distributed over $\mathbb{Z}_q^+$. More precisely, if

$$x \leftarrow \$ \mathbb{Z}_q^+, \quad y = \text{Compress}_q(x, d), \quad \tilde{x} \leftarrow \text{Inv}_q(y, d),$$

then $\tilde{x}$ is uniformly distributed over $\mathbb{Z}_q^+$. Indeed, for every $a \in \mathbb{Z}_q^+$, letting $y_a = \text{Compress}_q(a, d)$, we have

$$\Pr[\tilde{x} = a] = \Pr[y = y_a] \cdot \Pr[\tilde{x} = a \mid y = y_a] = \frac{|\text{Preim}_q(y_a, d)|}{q} \cdot \frac{1}{|\text{Preim}_q(y_a, d)|} = \frac{1}{q}.$$

Hence it follows an exact uniform distribution $\tilde{x} \leftarrow \$ \mathbb{Z}_q^+$. The same statements extend coefficient-wise to polynomials in $\mathcal{R}_q$ and to vectors over $\mathcal{R}_q$.

Error Correction Codes and Repetition Encoding. Unlike standard Module-LWR encryption schemes, `WEAVER` incorporates explicit error correction to tolerate substantially more aggressive compression. A binary Bose–Chaudhuri–Hocquenghem code is denoted by $\text{BCH}(n_c, k_c, t_c)$, where n_c is the block length, k_c is the message length, and t_c is the error-correcting capability.

For the highly noisy lower message layer, we further employ a 4-way repetition code combined with soft-decision decoding. Specifically, each lower-significance message bit is embedded into four distant polynomial coefficients at indices $i, i+stp, i+2 \cdot stp$, and $i+3 \cdot stp$. During decryption, instead of making hard decisions on individual coefficients, the receiver aggregates the absolute distances (L_1 norm) across all four dimensions before thresholding. This multi-dimensional soft-decision approach provides a substantial coding gain against the heavy rounding noise, which is then combined with the outer BCH decoding to achieve a cryptographically negligible decryption failure probability under the target parameter sets.

2.2 Data Encoding and Serialization

To transmit cryptographic objects (such as polynomials and vectors) over a network, they must be serialized into byte arrays. Furthermore, since our error correction codes operate on bits, we require explicit conversions between byte arrays and bit strings. We use $\mathcal{B}$ to denote the byte alphabet $\{0, \dots, 255\}$, and $\mathcal{B}^\ell$ to denote the set of byte strings of length ℓ .

BytesToBits and BitsToBytes. For a byte array $B \in \mathcal{B}^\ell$, the function $\text{BytesToBits}(B)$ outputs an array of 8ℓ bits. The conversion is done in little-endian order per byte: the i -th byte $B[i]$ is decomposed into 8 bits $b_{8i}, b_{8i+1}, \dots, b_{8i+7}$, where b_{8i} is the least significant bit of $B[i]$. Conversely, $\text{BitsToBytes}(\mathbf{b})$ takes an array of 8ℓ bits and packs every 8 bits into a byte in the same little-endian manner to output $B \in \mathcal{B}^\ell$. In our algorithms, when we slice a byte array (e.g., $\mu[0 : \ell_1/8 - 1]$), it implicitly assumes the array can be treated as a bit string via these functions.

Encode_d. Let $f \in R_{2^d}$ be a polynomial whose coefficients $f_0, f_1, \dots, f_{255}$ are integers in $\{0, \dots, 2^d - 1\}$. The function $\text{Encode}_d(f)$ serializes f into an array of $32 \cdot d$ bytes. The encoding process concatenates the d -bit representations of the coefficients in little-endian order. Specifically, we consider the sequence of $256 \cdot d$ bits formed by taking the least significant bit to the most significant bit of f_0 , followed by the bits of f_1 , and so on up to f_{255} . This bit sequence is then partitioned into blocks of 8 bits, where each block forms a byte.

Mathematically, let the output byte array be $B \in \mathcal{B}^{32d}$. The m -th bit of the byte array (where $m = 8 \cdot i + j$ for $i \in \{0, \dots, 32d - 1\}$ and $j \in \{0, \dots, 7\}$) corresponds to the y -th bit of the coefficient f_x , where $m = x \cdot d + y$ for $y \in \{0, \dots, d - 1\}$ and $x \in \{0, \dots, 255\}$.

When applied to a vector of polynomials $\mathbf{v} = (v_0, \dots, v_{k-1}) \in R^k$, $\text{Encode}_d(\mathbf{v})$ simply concatenates the byte arrays obtained by applying Encode_d to each polynomial sequentially:

$$\text{Encode}_d(\mathbf{v}) = \text{Encode}_d(v_0) \parallel \text{Encode}_d(v_1) \parallel \dots \parallel \text{Encode}_d(v_{k-1})$$

resulting in an array of $32 \cdot d \cdot k$ bytes.

Decode_d. The function $\text{Decode}_d(B)$ is the exact inverse of Encode_d . It takes an array of $32 \cdot d$ bytes and reconstructs a polynomial $f \in R$ with coefficients in $\{0, \dots, 2^d - 1\}$. It sequentially reads d bits from the byte array B (in little-endian order) to reconstruct the integer value of each coefficient $f_0, f_1, \dots, f_{255}$. Similarly, for a byte array representing a vector $\mathbf{v} \in R^k$, it consumes chunks of $32 \cdot d$ bytes to reconstruct each of the k polynomials sequentially.

An Example: $d = 12$. For the uncompressed polynomial ring $\mathcal{R}_q$ (where $q \leq 2^{12}$, e.g., $q = 3329$), we use $d = 12$. In this case, Encode_{12} takes a polynomial with coefficients in $[0, q - 1]$ and packs every two coefficients (f_{2i}, f_{2i+1}) into three bytes $(b_{3i}, b_{3i+1}, b_{3i+2})$ as follows:

$$\begin{aligned} b_{3i} &= f_{2i} \bmod 256 \\ b_{3i+1} &= \lfloor f_{2i}/256 \rfloor + ((f_{2i+1} \bmod 16) \cdot 16) \\ b_{3i+2} &= \lfloor f_{2i+1}/16 \rfloor \end{aligned}$$

This ensures efficient and standardized serialization for the secret key $\mathbf{s}$.

For the special case of $d = 12$, the function Decode_{12} acts as the exact inverse of Encode_{12} . It processes the input byte array B in chunks of three bytes $(b_{3i}, b_{3i+1}, b_{3i+2})$ to reconstruct the polynomial coefficients (f_{2i}, f_{2i+1}) as follow:

$$\begin{aligned} f_{2i} &= \lfloor b_{3i+1}/16 \rfloor \cdot 256 + b_{3i} \\ f_{2i+1} &= b_{3i+2} \cdot 16 + (b_{3i+1} \bmod 16) \end{aligned}$$

2.3 Algorithm Specification

2.3.1 Specification of IND-CPA-secure PKE

Description of the PKE Algorithms. The IND-CPA-secure Public Key Encryption (PKE) scheme follows the Module-LWR framework but introduces a randomized lifting mechanism (WEAVER-Inv) and a highly novel message encoding strategy to achieve exceptional noise tolerance.

Key Generation and Encryption (Algorithms 1 and 2): Unlike standard MLWE schemes that sample explicit error vectors, WEAVER-Inv adopts a pure rounding-based (MLWR-type) approach. The public matrix $\hat{\mathbf{A}}$ is generated directly in the NTT domain using an extendable-output function (XOF). The secret vector $\mathbf{s}$ and the ephemeral secret $\mathbf{r}$ are sampled from a centered binomial distribution (CBD). No explicit error vectors $(\mathbf{e}, \mathbf{e}', \mathbf{e}'')$ are added. Instead, cryptographic hardness is derived entirely from the aggressive truncation of the lower bits using the Compress_q function.

Crucially, during encryption, the sender does not reconstruct the public key using a deterministic decompression. Instead, it applies the randomized lifting map Inv_q to sample a uniformly random preimage $\hat{\mathbf{t}}$ from the compression bucket. To ensure the encryption process is strictly deterministic for a given random coin seed_1 —a strict prerequisite for applying the Fujisaki-Okamoto (FO) transform—the randomness used for this lifting is explicitly derived from seed_1 via a PRF. This randomized lifting is essential for the exact uniformity-preserving property relied upon in the security proof.

To align the encryption algorithm with the standard decisional MLWR reduction, the message is embedded in the compressed domain rather than before rounding. If the message were added before compression, the resulting distribution would take the form $\text{Compress}_q(x + w, d_v)$, which is not in general obtainable from the standard MLWR output $\text{Compress}_q(x, d_v)$ by an efficient exact transformation due to the non-linearity of the rounding map. Embedding the message after compression avoids this obstacle entirely, ensuring that the second ciphertext component is obtained from a standard MLWR sample via public deterministic post-processing.

Message Encoding (Algorithm 4): Unlike standard schemes that encode 1 bit per polynomial coefficient, our MsgEncode algorithm employs a *Multi-Level Coding* strategy combined with explicit error correction. The input message μ is split into two parts: $\bar{\mu}$ (destined for the higher bits) and $\check{\mu}$ (destined for the lower bits). Both parts are independently encoded using BCH codes to produce codewords $\tilde{\mu}$ and $\check{\mu}$, respectively. The encoded message is constructed in the compressed domain $\mathbb{Z}_{2^{d_v}}^n$:

- *Higher Bits Embedding:* The bits of $\tilde{\mu}$ are embedded into the most significant part of the compressed coefficients by multiplying them by 2^{d_v-1} .
- *Lower Bits Embedding (Repetition Code):* The bits of $\check{\mu}$ are embedded into the lower significant part by multiplying by 2^{d_v-2} . Crucially, to ensure robustness against high noise, each bit of $\check{\mu}$ is spread across four distant coefficients (indices $i, i + \text{stp}, i + 2 \cdot \text{stp}, i + 3 \cdot \text{stp}$). This 4-way repetition effectively maximizes the Euclidean distance between valid codewords in the multi-dimensional space.

Message Decoding and Deterministic Decryption (Algorithms 5 and 6): For decryption, the receiver simply uses the standard deterministic canonical decompression Decompress_q to lift the ciphertext components. Because the decompression map naturally scales the compressed-domain offsets (2^{d_v-1} and 2^{d_v-2}) back to the ring $\mathcal{R}_q$ (approximately $\frac{q}{2}$ and $\frac{q}{4}$), the subsequent decoding logic remains completely oblivious to the compressed-domain embedding.

After lifting, the decryption process computes the noisy encoded message $w' \approx w_{\text{ideal}} + \text{noise}$. To reliably recover μ from w' , MsgDecode utilizes a *Successive Interference Cancellation (SIC)* approach:

1. *Extracting Lower Bits:* The algorithm first isolates the lower bits by evaluating the coefficients modulo $\frac{q-1}{2}$. This operation elegantly nullifies the interference from the higher bits. It then aggregates the absolute distance (L_1 norm) from the expected value $\lfloor q/4 \rfloor$ across the four repetition positions. A soft-decision rule is applied: if the accumulated error is large ($\geq \frac{q-1}{2}$), the bit is decoded as 0; otherwise, 1.
2. *First-stage Error Correction:* The extracted lower bits μ' are passed through the BCH decoder to correct any remaining errors, yielding the exact original lower message $\dot{\mu}$.
3. *Interference Cancellation:* The corrected lower bits are re-encoded to reconstruct the exact lower-bit interference $\ddot{\mu}$, which is then explicitly subtracted from the noisy polynomial w' to obtain $\bar{w}$.
4. *Extracting Higher Bits:* With the lower-bit interference completely removed, the higher bits can now be safely extracted using standard rounding $\lceil (2/q)\bar{w}_i \rceil$.
5. *Second-stage Error Correction:* Finally, the extracted higher bits are passed through the second BCH decoder to recover $\bar{\mu}$.

Algorithm 1 WEAVER.CPAPKE.KeyGen(1^λ): Key Pair Generation

Output: Secret key $sk = \mathbf{s} \in \mathcal{R}_q^k \rightarrow \mathcal{B}^{12 \cdot k \cdot n/8}$

Output: Public key $pk = (\mathbf{t} \in \mathcal{R}_q^k, seed_A) \rightarrow \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

```

1:  $seed_0 \leftarrow \$_{\mathcal{B}^{4\lambda/8}}$ 
2:  $(seed_A, seed_{sk}) = \text{XOF}(seed_0)$   $\triangleright seed_A, seed_{sk} \in \mathcal{B}^{4\lambda/8}$ 
3: for  $i$  from 0 to  $k - 1$  do
4:   for  $j$  from 0 to  $k - 1$  do
5:      $\hat{\mathbf{A}}[i][j] \leftarrow \text{XOF}(seed_A, i, j)$   $\triangleright$  Generate matrix  $\hat{\mathbf{A}} \in \mathcal{R}_q^{k \times k}$  in NTT domain
6:   end for
7: end for
8:  $ctr = 0$ 
9: for  $i$  from 0 to  $k - 1$  do
10:   $\mathbf{s}[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(seed_{sk}, ctr))$   $\triangleright$  Sample  $\mathbf{s} \in \mathcal{R}_q^k$ 
11:   $ctr = ctr + 1$ 
12: end for
13:  $\hat{\mathbf{s}} = \text{NTT}(\mathbf{s})$ 
14:  $\mathbf{t} = \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \hat{\mathbf{s}})$   $\triangleright$  Pure MLWR: no explicit error  $\mathbf{e}$  is added
15:  $sk = \text{Encode}_{12}(\hat{\mathbf{s}})$ 
16:  $pk = (\text{Encode}_{d_t}(\text{Compress}_q(\mathbf{t}, d_t)), seed_A)$ 
17: return  $(pk, sk)$ 

```

Algorithm 2 WEAVER.CPAPKE.Enc($pk, \mu; seed_1$): Encryption

Input: Public key $pk \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

Input: Message $\mu \in \mathcal{B}^{n/8}$

Input: Random coins $seed_1 \in \mathcal{B}^{4\lambda/8}$

Output: Ciphertext $ct = (\mathbf{u}, c_2) \in (\mathcal{R}_q^k, \mathbb{Z}_{2^{d_v}}^n) \rightarrow \mathcal{B}^{(d_u \cdot k + d_v)n/8}$

```

1:  $(\mathbf{t}', seed_A) = \text{Parse}(pk)$  ▷ Decode public key and seed
2:  $ctr = 0$ 
3:  $\mathbf{t}_{\text{lift}} = \text{Inv}_q(\mathbf{t}', d_t; \text{PRF}(seed_1, ctr))$  ▷ Derandomized lifting using  $seed_1$ 
4:  $ctr = ctr + 1$ 
5:  $\hat{\mathbf{t}} = \text{NTT}(\mathbf{t}_{\text{lift}})$ 
6: for  $i$  from 0 to  $k - 1$  do
7:   for  $j$  from 0 to  $k - 1$  do
8:      $\hat{\mathbf{A}}^T[i][j] \leftarrow \text{XOF}(seed_A, j, i)$  ▷ Generate the transpose of matrix  $\hat{\mathbf{A}}$ 
9:   end for
10: end for
11: for  $i$  from 0 to  $k - 1$  do
12:    $\mathbf{r}[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(seed_1, ctr))$  ▷ Sample  $\mathbf{r} \in \mathcal{R}_q^k$ 
13:    $ctr = ctr + 1$ 
14: end for
15:  $\hat{\mathbf{r}} = \text{NTT}(\mathbf{r})$ 
16:  $w = \text{MsgEncode}(\mu)$  ▷  $w \in \mathbb{Z}_{2^{d_v}}^n$ 
17:  $\mathbf{u} = \text{NTT}^{-1}(\hat{\mathbf{A}}^T \circ \hat{\mathbf{r}})$  ▷ No explicit error  $\mathbf{e}'$ 
18:  $v_0 = \text{NTT}^{-1}(\hat{\mathbf{t}}^T \circ \hat{\mathbf{r}})$  ▷ No explicit error  $\mathbf{e}''$ 
19:  $c_1 = \text{Encode}_{d_u}(\text{Compress}_q(\mathbf{u}, d_u))$ 
20:  $c_2 = \text{Encode}_{d_v}((\text{Compress}_q(v_0, d_v) + w) \bmod 2^{d_v})$  ▷ Add message in compressed domain
21: return  $ct = (c_1 \parallel c_2)$ 

```

Algorithm 3 Inv_q($\mathbf{t}', d_t; seed$): Encryption

Input: Compressed public key $\mathbf{t}' \in \mathcal{R}_{2^{d_t}}^k$

Input: Decompress parameter d_t

Input: Random coins $seed \in \mathcal{B}^{4\lambda/8}$

Output: Re-randomized decompressed public key $\mathbf{t}_{\text{lift}} \in \mathcal{R}_q^k$

```

1: for  $i$  from 0 to  $k - 1$  do
2:   for  $j$  from 0 to  $n - 1$  do
3:      $e \leftarrow \$ \text{Preim}_q^\pm(\mathbf{t}'^{(i)}[j], d_t)$  ▷ sample uniformly from error bucket, using  $seed$ 
4:      $\mathbf{t}_{\text{lift}}^{(i)}[j] = \text{Decompress}_q(\mathbf{t}'^{(i)}[j], d_t) + e$ 
5:   end for
6: end for

```

Algorithm 4 MsgEncode(μ): Message Encoding

Input: Message bits $\mu \in \mathcal{B}^{n/8}$

Output: Encoded element $w \in \mathbb{Z}_{2^{d_v}}^n$

```

1:  $\bar{\mu} := \mu[0 : \ell_1/8 - 1]$  ▷ Message bits to be encoded to Higher bits
2:  $\dot{\mu} := \mu[\ell_1/8 : n/8 - 1]$  ▷ Message bits to be encoded to Lower bits
3:  $\tilde{\mu} = \text{ECCEncode}_1(\bar{\mu})$  ▷ getting codeword from BCH
4:  $\ddot{\mu} = \text{ECCEncode}_2(\dot{\mu})$  ▷ getting codeword from BCH
5:  $w = 0 \in \mathbb{Z}_{2^{d_v}}^n$ 
6: for  $i$  from 0 to  $n - 1$  do ▷ Encode to Higher bits in compressed domain
7:    $w_i = w_i + 2^{d_v-1} \cdot \tilde{\mu}_i$ 
8: end for
9: for  $i$  from 0 to  $\ell_3 - 1$  do ▷ Encode to Lower bits in compressed domain
10:   $w_i = w_i + 2^{d_v-2} \cdot \ddot{\mu}_i$ 
11:   $w_{i+stp} = w_{i+stp} + 2^{d_v-2} \cdot \ddot{\mu}_i$ 
12:   $w_{i+2 \cdot stp} = w_{i+2 \cdot stp} + 2^{d_v-2} \cdot \ddot{\mu}_i$ 
13:   $w_{i+3 \cdot stp} = w_{i+3 \cdot stp} + 2^{d_v-2} \cdot \ddot{\mu}_i$ 
14: end for
15: return  $w$ 

```

Algorithm 5 WEAVER.CPAPKE.Dec(sk, ct): Decryption

Input: Secret key $sk \in \mathcal{B}^{12 \cdot k \cdot n/8}$

Input: Ciphertext $ct \in \mathcal{B}^{(d_u \cdot k + d_v) \cdot n/8}$

Output: Message $\mu \in \mathcal{B}^{n/8}$

```

1:  $(c_1, c_2) := \text{Parse}(ct)$ 
2:  $\mathbf{u} = \text{Decompress}_q(\text{Decode}_{d_u}(c_1), d_u)$  ▷ Deterministic canonical decompression
3:  $\mathbf{v} = \text{Decompress}_q(\text{Decode}_{d_v}(c_2), d_v)$  ▷ Automatically scales message back to  $\mathcal{R}_q$ 
4:  $\hat{\mathbf{s}} = \text{Decode}_{12}(sk)$ 
5:  $w' = \mathbf{v} - \text{NTT}^{-1}(\hat{\mathbf{s}}^T \circ \text{NTT}(\mathbf{u})) \bmod^+ q$ 
6:  $\mu = \text{MsgDecode}(w')$ 
7: return  $\mu$ 

```

Algorithm 6 MsgDecode(w): Message Decoding

Input: Decrypted ring element $w \in \mathcal{R}_q$ **Output:** Decoded Message bits $\mu \in \mathcal{B}^{n/8}$

```
1: for  $i$  from 0 to  $\ell_3 - 1$  do                                     ▷ Decode from Lower bits
2:    $ee = \left\lfloor w_i \bmod^+ \frac{q-1}{2} - \lfloor q/4 \rfloor \right\rfloor$ 
3:    $ee = ee + \left\lfloor w_{i+stp} \bmod^+ \frac{q-1}{2} - \lfloor q/4 \rfloor \right\rfloor$ 
4:    $ee = ee + \left\lfloor w_{i+2 \cdot stp} \bmod^+ \frac{q-1}{2} - \lfloor q/4 \rfloor \right\rfloor$ 
5:    $ee = ee + \left\lfloor w_{i+3 \cdot stp} \bmod^+ \frac{q-1}{2} - \lfloor q/4 \rfloor \right\rfloor$ 
6:    $\mu'_i = (ee \geq \frac{q-1}{2}) ? 0 : 1$ 
7: end for
8:  $\dot{\mu} = \text{ECCDecode}_2(\mu')$                                      ▷ Decoding BCH
9:  $\ddot{\mu} = \text{ECCEncode}_2(\dot{\mu})$ 
10:  $\bar{w} = w$ 
11: for  $i$  from 0 to  $\ell_3 - 1$  do                                     ▷ Remove Lower bits
12:    $\bar{w}_i = \bar{w}_i - \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
13:    $\bar{w}_{i+stp} = \bar{w}_{i+stp} - \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
14:    $\bar{w}_{i+2 \cdot stp} = \bar{w}_{i+2 \cdot stp} - \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
15:    $\bar{w}_{i+3 \cdot stp} = \bar{w}_{i+3 \cdot stp} - \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
16: end for
17: for  $i$  from 0 to  $n - 1$  do                                     ▷ Decode from Higher bits
18:    $\mu''_i = \lceil (2/q) \cdot \bar{w}_i \rceil$ 
19: end for
20:  $\bar{\mu} = \text{ECCDecode}_1(\mu'')$                                      ▷ Decoding BCH
21: return  $\mu := (\bar{\mu} \parallel \dot{\mu})$ 
```

2.3.2 Specification of IND-CCA-secure KEM

To achieve IND-CCA security, we apply a variant of the Fujisaki-Okamoto (FO) transform to our underlying IND-CPA-secure PKE scheme. This transform ensures that any modification to the ciphertext will result in a pseudorandom shared secret, providing implicit rejection of invalid ciphertexts. We utilize three cryptographic hash functions modeled as random oracles: H (e.g., SHA3-256), G (e.g., SHA3-512), and J (e.g., SHAKE-256).

KEM Algorithms. The KEM consists of three algorithms: `WEAVER.CCAKEM.KeyGen`, `WEAVER.CCAKEM.Encaps`, and `WEAVER.CCAKEM.Decaps`, which are detailed in Algorithms 7, 8, and 9, respectively. The byte length of both PKE's encrypted message and KEM's shared secret equals $\text{len}_{ss} = |\mu|/8$. **The byte length of hash output len_H ...** **The byte length of public key (matrix A) seed len_{seed} ...**

Algorithm 7 `WEAVER.CCAKEM.KeyGen`(1^λ): KEM Key Generation

Output: KEM Secret key $sk_{KEM} \in \mathcal{B}^{12 \cdot k \cdot n/8 + d_t \cdot k \cdot n/8 + \text{len}_H + \text{len}_{ss}}$

Output: KEM Public key $pk \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

- 1: $z \leftarrow \$ \mathcal{B}^{\text{len}_{ss}}$ ▷ Sample implicit rejection value
 - 2: $(pk, sk_{PKE}) = \text{WEAVER.CPAPKE.KeyGen}(1^\lambda)$
 - 3: $sk_{KEM} = (sk_{PKE} \parallel pk \parallel H(pk) \parallel z)$
 - 4: **return** (pk, sk_{KEM})
-

Algorithm 8 `WEAVER.CCAKEM.Encaps`(pk): KEM Encapsulation

Input: KEM Public key $pk \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

Output: Ciphertext $ct \in \mathcal{B}^{(d_u \cdot k + d_v) \cdot n/8}$

Output: Shared secret $K \in \mathcal{B}^{\text{len}_{ss}}$

- 1: $m \leftarrow \$ \mathcal{B}^{\text{len}_{ss}}$ ▷ Sample random message
 - 2: $(K, r) = G(m \parallel H(pk))$ ▷ Derive shared secret and coins
 - 3: $ct = \text{WEAVER.CPAPKE.Enc}(pk, m; r)$ ▷ Encrypt using derived coins
 - 4: **return** (ct, K)
-

Algorithm 9 `WEAVER.CCAKEM.Decaps`(sk_{KEM}, ct): KEM Decapsulation

Input: KEM Secret key $sk_{KEM} = (sk_{PKE} \parallel pk \parallel h_{pk} \parallel z)$

Input: Ciphertext $ct \in \mathcal{B}^{(d_u \cdot k + d_v) \cdot n/8}$

Output: Shared secret $K \in \mathcal{B}^{\text{len}_{ss}}$

- 1: $m' = \text{WEAVER.CPAPKE.Dec}(sk_{PKE}, ct)$ ▷ Decrypt the ciphertext
 - 2: $(K, r') = G(m' \parallel h_{pk})$ ▷ Re-derive shared secret and coins
 - 3: $ct' = \text{WEAVER.CPAPKE.Enc}(pk, m'; r')$ ▷ Re-encrypt to verify ciphertext
 - 4: **if** $ct \neq ct'$ **then**
 - 5: $K = J(z, ct)$ ▷ Invalid ciphertext: implicit rejection and overwrite output secret
 - 6: **end if**
 - 7: **return** K
-

Explanation of the FO Transform. The KEM Key Generation algorithm extends the PKE secret key by appending the public key pk , its hash $H(pk)$, and a uniformly random 32-byte value z . The value z is crucial for the implicit rejection mechanism. During Encapsulation, a random 32-byte message m is chosen and hashed. The function G acts as a random oracle to derive both a pre-master secret $\hat{K}$ and the deterministic randomness r required for the PKE encryption. The

final shared secret K is bound to the ciphertext by hashing ct into the KDF. In Decapsulation, the receiver decrypts the ciphertext to obtain m' and re-runs the encryption process using the re-derived randomness r' . If the re-computed ciphertext ct' matches the received ct , the decapsulation succeeds, and the valid shared secret is returned. If the check fails (indicating a potentially malicious or malformed ciphertext), the KEM does not abort but instead returns a pseudorandom key derived from the secret value z and the hash of the ciphertext. This implicit rejection prevents chosen-ciphertext attacks (CCA) from leaking information about the secret key.

2.4 Parameter Sets

To meet the diverse security requirements of different applications, we propose three parameter sets for WEAVER, targeting NIST security categories 1, 3, and 5 (corresponding to AES-128, AES-256, and AES-512, respectively). We denote these instances as WEAVER-512, WEAVER-1024, and WEAVER-2048.

Core Parameters. Table 1 lists the core cryptographic parameters. All parameter sets share the same polynomial ring modulus $q = 3329$. The ring dimension n and module rank k scale according to the targeted security level. The noise parameter η_1 define the width of the centered binomial distributions (CBD) used for sampling the secret. The error vectors are sampled deterministically from rounding. To minimize bandwidth, the public key and ciphertext components are aggressively compressed to d_t, d_u , and d_v bits.

Table 1: Proposed Core Parameter Sets for WEAVER

Instance	Level	$ \mu $	n	k	q	η_1	(d_t, d_u, d_v)
WEAVER-512	1	128	256	2	3329	5	(9, 9, 4)
WEAVER-1024	3	256	256	4	3329	2	(9, 9, 5)
WEAVER-2048	5	512	512	4	3329	1	(9, 9, 6)

Error Correction and Decryption Failure Rate (DFR). A defining feature of WEAVER is its robust multi-level error correction mechanism, detailed in Table 2. The shared secret μ is split into two parts: $\bar{\mu}$ (encoded in the higher bits) and $\dot{\mu}$ (encoded in the lower bits via a D_4 lattice).

- For WEAVER-512, the noise margin is sufficient to encode the entire 128-bit message in the higher bits using a single BCH layer.
- For higher security levels (WEAVER-1024 and WEAVER-2048), we utilize the two-layer encoding. The lower bits are protected by a combination of BCH codes and D_4 lattice repetition, while the higher bits are protected by another BCH code.

The overall decryption failure rate δ is bounded by the maximum failure probability of the two layers, i.e., $\delta = \max(\delta^{(1)}, \delta^{(2)})$. As shown in Table 2, all parameter sets achieve a cryptographically negligible DFR (e.g., $\leq 2^{-134}$).

Effective LWE Noise and Security Amplification. When evaluating the concrete security of our scheme against lattice reduction attacks (e.g., primal and dual attacks), we consider the *effective noise* introduced by both the explicit error sampling and the deterministic rounding operations.

Table 2: Error Correction Parameters and Decryption Failure Rates

Parameter	WEAVER-512	WEAVER-1024	WEAVER-2048
Total Message Bits $ \mu $	128	256	512
Encoding Strategy	Layer 1 only	Layer 1 + Layer 2	Layer 1 + Layer 2
Layer 1 (Higher Bits)			
Message length $\ell_1 = \bar{\mu} $	128	223	456
BCH Code (n, k, t)	(255, 128, 5)	(255, 223, 4)	(511, 456, 6)
Layer 1 DFR ($\log_2 \delta^{(1)}$)	-181.8	-189.2	-261.9
Layer 2 (Lower Bits / D_4)			
Message length $\ell_2 = \dot{\mu} $	0	33	56
BCH Code (n, k, t)	-	(63, 33, 4)	(127, 85, 6)
Codeword length ℓ_3	-	57	98
step length (stp)	-	64	128
Layer 2 DFR ($\log_2 \delta^{(2)}$)	-	-183.7	-269.9
Overall DFR ($\log_2 \delta$)	-181.8	-183.7	-261.9

In our scheme, the decompression operation introduces a rounding error e_{round} that is uniformly distributed in the interval $[-\frac{q}{2^{d+1}}, \frac{q}{2^{d+1}}]$. Therefore, the variance of the effective noise is: $\sigma_{\text{eff}}^2 \approx \frac{q^2}{12 \cdot 2^{2d}}$.

This phenomenon, closely related to the Learning With Rounding (LWR) problem, provides a significant security amplification. While schemes like ML-KEM also benefit from this, their ability to leverage rounding error is strictly bottlenecked by the need to maintain a negligible DFR without explicit error correction. In contrast, our robust BCH and D_4 -lattice error correction mechanism allows us to employ highly aggressive compression parameters (i.e., smaller d_t, d_u, d_v). We intentionally inject a much larger σ_{round}^2 into the LWE instances. This design rationale elegantly achieves two goals simultaneously: it drastically reduces the bandwidth requirements, while simultaneously boosting the core-SVP hardness.

Table 3 summarizes the Core-SVP security estimates (accounting for the effective noise) and the resulting sizes for the public key (pk) and ciphertext (ct). Our parameter sets comfortably exceed the security thresholds of their respective NIST categories while maintaining highly competitive bandwidth profiles.

Table 3: Core-SVP Security Estimates and Bandwidth

Instance	Classical Security	Quantum Security	$ pk $ (Bytes)	$ ct $ (Bytes)
WEAVER-512	130	-	608	736(-55)
WEAVER-1024	257	-	1184	1312
WEAVER-2048	517	-	2336	2688

3 Design Rationale

3.1 Underlying Hard Problems

The security of WEAVER is rooted in the hardness of lattice problems over module lattices, specifically the Learning With Rounding (LWR) problem and its module variant (MLWR). Unlike traditional lattice-based schemes that rely on the addition of explicit Gaussian or binomial noise, WEAVER adopts a pure rounding-based approach to derive its cryptographic hardness.

This design choice is motivated by the need for extreme bandwidth efficiency and the theoretical elegance of the randomized lifting mechanism.

Let $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$ be the polynomial ring, and let k denote the module rank. The core security of the scheme is analyzed through the lens of the decisional Module Learning With Rounding (MLWR) problem.

Module Learning With Rounding (MLWR). The decisional $\text{MLWR}_{k,\ell,\eta,d}$ problem asks an adversary to distinguish between the following two distributions:

1. *Uniform distribution:* $\mathbf{A} \leftarrow \$ \mathcal{R}_q^{\ell \times k}$ and $\mathbf{u} \leftarrow \$ \mathcal{R}_q^\ell$ are sampled uniformly at random, and the adversary receives $(\mathbf{A}, \text{Compress}_q(\mathbf{u}, d))$;
2. *MLWR distribution:* $\mathbf{A} \leftarrow \$ \mathcal{R}_q^{\ell \times k}$, $\mathbf{s} \leftarrow \$ \text{CBD}_\eta^k$, and the adversary receives $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s}, d))$.

In WEAVING, the public key is a compressed MLWR sample with $\ell = k$, and the ciphertext components are compressed MLWR samples with $\ell = k + 1$. The hardness of MLWR over rings with polynomial moduli is a well-studied problem. As demonstrated by Chen, Zhang, and Zhang [CZZ18], the hardness of such instances can be rigorously supported by the Computational Ring-LWR (R-CLWR) framework, which provides a reduction from worst-case lattice problems even when the modulus q is not exponentially large.

The Role of Randomized Lifting in Security. A significant challenge in compressed lattice schemes is the distribution of the public key during the encryption phase. In standard designs, the sender must use a deterministic decompression map $\text{Decompress}_q(\cdot, d)$, which produces a lifted vector that is restricted to a small subset of $\mathcal{R}_q^k$, thereby breaking the uniformity required for a tight reduction to standard MLWR.

WEAVING solves this problem via the **randomized lifting** map $\text{Inv}_q(\cdot, d)$. As established in Section 2.1, this map satisfies the **exact uniformity-preserving property**: if pk is a uniformly random compressed value, then $\tilde{\mathbf{t}} \leftarrow \$ \text{Inv}_q(pk, d)$ is distributed exactly uniformly over $\mathcal{R}_q^k$. This property is the cornerstone of our security proof in Section 4.1, as it allows the simulator to perfectly restore the uncompressed MLWR structure during the game-hopping sequence without introducing any statistical divergence.

Effective Noise Interpretation for Concrete Security. While the theoretical proof is anchored in the MLWR assumption, for the purpose of concrete security estimation (Section ??), we adopt a noise-centric viewpoint. The deterministic rounding operation $\text{Compress}_q(\cdot, d)$ inherently introduces a rounding error that is approximately uniform over the quantization interval. In our analysis, this rounding error is treated as the **effective noise** of the LWE instance. The variance of this noise is given by:

$$\sigma_{\text{eff}}^2 \approx \frac{q^2}{12 \cdot 2^{2d}}.$$

By aggressively reducing the compression precision d (e.g., $d_v = 4$), WEAVING intentionally injects a large amount of rounding noise into the system. This design philosophy, which we term “Dancing with Imperfection,” achieves two goals simultaneously: it drastically reduces the communication overhead and significantly boosts the concrete Core-SVP hardness against lattice reduction attacks.

Security Rationale Summary. In summary, the security of WEAVING is justified at three complementary levels:

- **Provable Level:** The scheme reduces directly to the standard decisional MLWR problem, facilitated by the randomized lifting mechanism.

- **Hardness Level:** The underlying MLWR instances are supported by the R-CLWR framework [CZZ18], ensuring a connection to worst-case lattice problems.
- **Concrete Level:** The scheme is evaluated against the best known primal and dual attacks using the effective noise magnitude induced by aggressive compression.

This multi-layered rationale ensures that WEAVER provides robust post-quantum security while reaching the limits of bandwidth optimization.

3.2 Message Encoding Strategy and Error Correction

The most significant departure of Weaver from standard Module-LWE based KEMs (like Kyber/ML-KEM) lies in its highly optimized message encoding mechanism. To achieve extreme compactness in ciphertext and public key sizes, Weaver aggressively compresses the transmitted elements. This aggressive compression naturally introduces a much higher noise variance during decryption. To safely manage this noise and ensure a negligible Decryption Failure Rate (DFR), we introduce a novel **Hierarchical Encoding Scheme with Concatenated D4-Lattice and BCH Codes**.

Layer 1: Most Significant Bits (MSB) Encoding. The first part of the message $\bar{\mu}$ is encoded into the most significant bits of the polynomial coefficients. Since the MSBs are relatively robust against the LWE error term, the error rate before correction is moderate. We employ a standard Bose-Chaudhuri-Hocquenghem (BCH) code (e.g., BCH1) to correct these residual errors, bringing the failure probability of this layer down to cryptographic insignificance.

Layer 2: Next-Most Significant Bits Encoding via D4 Lattice. To embed sufficient key material (e.g., 256 bits) without increasing the polynomial ring dimension n , the remaining message bits $\hat{\mu}$ must be encoded into the next-most significant bits. However, these bits suffer from a significantly higher error rate due to the accumulated LWE noise and compression rounding errors. Simple scalar rounding is insufficient here. To overcome this, we utilize the **D4 Lattice** for multi-dimensional quantization. By spreading 1 bit of information across 4 coefficients (as seen in our `MsgEncode` algorithm), we leverage the superior sphere-packing density of the D4 lattice. This multi-dimensional approach provides a substantial coding gain against Gaussian-like noise. To eliminate any remaining errors after D4 closest-point decoding, we concatenate it with a secondary BCH code (BCH2). The synergy of D4 lattice quantization and BCH error correction allows Weaver to extract reliable bits from highly noisy coefficients.

3.3 Parameter Selection Strategy

The parameter selection for WEAVER is formulated as a multi-objective optimization problem. Our primary goal is to minimize the total bandwidth (public key and ciphertext sizes) while strictly satisfying two constraints: (1) The Core-SVP hardness must meet or exceed the targeted NIST security categories (Levels 1, 3, and 5). (2) The average-case Decryption Failure Rate (DFR) must remain cryptographically negligible (e.g., $\leq 2^{-128}$).

To achieve this optimal balance, we decoupled the parameter selection into four distinct categories: algebraic structure, security scaling, bandwidth compression, and error correction.

3.3.1 Algebraic Parameters (n, q)

For the underlying polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$, we selected $n = 256$ (and $n = 512$ for Level 5) and the prime modulus $q = 3329$. The choice of $q = 3329$ is highly deliberate. It is a prime congruent to 1 (mod 256), which allows for a fully split Number Theoretic Transform (NTT) over the ring. This specific prime has been extensively vetted by the cryptographic

community (e.g., in ML-KEM/Kyber) and enables highly optimized arithmetic implementations across various hardware and software platforms. Keeping n as a power of 2 ensures the algebraic simplicity of the cyclotomic ring.

3.3.2 Security Scaling Parameters (k, η_1, η_2)

The module rank k serves as the primary coarse-grained knob for scaling the security level. We use $k = 2$ for Category 1, and $k = 4$ for Categories 3 and 5. To fine-tune the security and control the initial LWE noise, we utilize the Centered Binomial Distribution (CBD). The parameters η_1 (for secret keys) and η_2 (for error vectors) are chosen to be small (ranging from 1 to 4). A smaller η reduces the raw noise variance, leaving more “noise budget” for our aggressive compression step, while still providing sufficient entropy to thwart combinatorial and algebraic attacks.

3.3.3 Bandwidth Compression Parameters (d_t, d_u, d_v)

The most distinctive aspect of WEAVER’s parameter strategy is the aggressive truncation of the public key and ciphertext components. We drop a significant number of lower bits, retaining only d_t, d_u , and d_v bits for $\mathbf{t}, \mathbf{u}$, and v , respectively. Unlike standard schemes that conservatively choose these values to avoid decryption failures, we intentionally push d_t, d_u, d_v to their extreme lower limits (e.g., $d_v = 4$). This drastically reduces the serialized sizes. The massive rounding error introduced by this aggressive compression acts as an additional source of cryptographic noise, effectively amplifying the concrete security against lattice reduction attacks (as discussed in Section 4).

3.3.4 Error Correction Parameters (BCH and D_4 Lattice)

The aggressive compression strategy inevitably results in a raw noise variance that far exceeds the standard correctable threshold of pure rounding-based decoding. To resolve this, we parameterize our hierarchical error correction mechanism to perfectly absorb this excess noise.

- **Message Splitting:** The shared secret is divided into higher bits ($\bar{\mu}$) and lower bits ($\dot{\mu}$). The lengths ℓ_1 and ℓ_2 are dynamically allocated based on the noise capacity of the specific security level.
- **D_4 Repetition:** For the highly noisy lower bits, we utilize a 4-way repetition mapped to a D_4 lattice structure. This multi-dimensional quantization provides a significant coding gain, artificially lowering the raw bit error rate.
- **BCH Capacity (t):** We select BCH codes with an error-correcting capability of $t = 3$ or $t = 4$. This specific capacity was mathematically derived: it is the exact threshold required to pull the subgaussian tail of the residual errors down to $\leq 2^{-134}$, ensuring perfect correctness in the average case without introducing excessive parity-check overhead.

By jointly optimizing these four sets of parameters, WEAVER achieves a Pareto-optimal trade-off: it leverages heavy error correction to tolerate extreme compression, resulting in highly compact ciphertexts while maintaining robust security and correctness.

3.4 Decryption Failure Analysis

In this subsection, we analyze the decryption correctness of WEAVER in the honest-execution setting. Our goal is to justify the parameter choices and the use of the two-layer decoding mechanism by estimating the average-case decryption failure rate (DFR) under randomly generated keys, messages, and encryption randomness.

3.4.1 Layered Decoding Thresholds

The decoding procedure of WEAVER is hierarchical. As described in Algorithm 6, the lower-layer bits and upper-layer bits are recovered through different decision rules.

The purpose of the two-layer coding strategy is precisely to convert these moderate raw error probabilities into a negligible final DFR.

For the higher-bit layer, the BCH code corrects up to t_1 bit errors in each codeword. If the raw coefficient error probability is p_{high} , then the residual failure probability of this layer is bounded by the upper tail of the corresponding binomial distribution.

For the lower-bit layer, the D_4 repetition first reduces the effective bit error probability, and the outer BCH code then corrects the remaining residual errors up to its correction capability t_2 . This produces a second-layer failure probability that is likewise negligible.

Thus, the decryption correctness should be analyzed separately for:

- the higher-bit layer (Layer 1) protected by BCH decoding, and
- the lower-bit layer (Layer 2) protected by D_4 repetition followed by BCH decoding.

3.4.2 Single Bit Failure Rate

We first state the following lemmas about single bit failure rate (BFR) at each layer. Then we will use them to deduce the overall decryption failure rate (DFR) of our scheme.

According to Algorithm 5, the receiver computes

$$w' = v - \text{NTT}^{-1}(\hat{\mathbf{s}}^T \circ \text{NTT}(\mathbf{u})). \quad (1)$$

Let $\mathbf{c}_t$, $\mathbf{c}_u$, and c_v denote the distortions introduced by compression and decompression. Substituting the encryption equations from Algorithm 2, the raw decryption noise polynomial $e_{\text{raw}} = w' - w$ can be written as

$$e_{\text{raw}} = \mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + c_v. \quad (2)$$

Before looking into the details, we first define some error distributions needed in our failure rate analysis. First we define:

Definition 3.1 (Compression Error). *Let ψ_d^k be the following distribution over $\mathcal{R}_q^k$:*

1. Choose uniformly random $\mathbf{x} \leftarrow \$ \mathcal{R}_{q^+}^k$
2. return $(\text{Decompress}_q(\text{Compress}_q(\mathbf{x}, d), d) - \mathbf{x}) \bmod^{\pm} q$.

Definition 3.2 (Asymmetric Compression Error). *Let $\bar{\psi}_d^k$ be the following distribution over $\mathcal{R}_q^k$:*

1. Choose uniformly random $\mathbf{x} \leftarrow \$ \mathcal{R}_{q^+}^k$
2. return $(\text{Decompress}_q(\text{Compress}_q(\mathbf{x}, d) + \frac{2d}{2}, d) - \mathbf{x} - \frac{q-1}{2}) \bmod^{\pm} q$.

Definition 3.3 (Randomized Lifting Error). *Let φ_d^k be the following distribution over $\mathcal{R}_q^k$:*

1. Choose uniformly random $\mathbf{x} \leftarrow \$ \mathcal{R}_{q^+}^k$
2. Choose uniformly random $\mathbf{e} \leftarrow \$ \text{Preim}_q^{\pm}(\mathbf{x}, d)$
3. return $\text{Inv}_q(\text{Compress}_q(\mathbf{x}, d), d) - \mathbf{x} \bmod^{\pm} q$, which is equivalent to returning $\text{Decompress}_q(\text{Compress}_q(\mathbf{x}, d), d) - \mathbf{x} + \mathbf{e} \bmod^{\pm} q$.

The higher-bit layer message embedding has followed the common practice, namely we have:

Lemma 3.4 (Layer 1 BFR; modified from [BDK⁺18]). *Let k be a positive integer, q be an odd positive integer. Let $\mathbf{s}, \mathbf{r}$ be random variables that have the same distribution as in Algorithms 1 and 2. Also, let $\mathbf{c}_t \leftarrow \varphi_{d_t}^k, \mathbf{c}_u \leftarrow \psi_{d_u}^k, \mathbf{c}_v \leftarrow \psi_{d_v}^k$. Denote: $\delta_0^{(1)} = \Pr[|(\mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + \mathbf{c}_v)_i| \geq \lfloor q/4 \rfloor]$, then the **single bit failure rate** at layer 1 is $\delta_0^{(1)}$.*

For the lower-bit layer, the message is embedded near $\lfloor \frac{q}{4} \rfloor$, and decoded after reduction modulo $\frac{q-1}{2}$. Before error correction, the corresponding scalar decision threshold is smaller, but each lower-layer bit is repeated over four coefficients in a D_4 -structured pattern. This repetition has substantially leveraged the failure threshold B_{dec} , thus improving robustness against noise.

Lemma 3.5 (Layer 2 BFR). *Let all symbols be the same as in the previous lemma. Let $B_{dec} = (4 \cdot \lfloor q/4 \rfloor)/2 = (q-1)/2$, and let the i -th coefficient of the decryption error polynomial be $e_{\text{raw}}[i] = (\mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + \mathbf{c}_v)_i$, $stp = 64/128$ for WEAVER-1024 and WEAVER-2048 respectively. Denote:*

$$\delta_0^{(2)} = \Pr \left[|e_{\text{raw}}[i]| + |e_{\text{raw}}[i + stp]| + |e_{\text{raw}}[i + 2 \cdot stp]| + |e_{\text{raw}}[i + 3 \cdot stp]| \geq B_{dec} \right],$$

then the **single bit failure rate after D_4 decoding** at layer 2 is $\delta_0^{(2)}$.

3.4.3 Independence Assumption and Overall Decryption Failure Rate

Proposition 3.6. *For a middle-result bit string of length ℓ with BFR δ_0 , if δ_0 is small enough, the estimation of its DFR δ after corrects up to t -bits error (i.e. the number of flipped bits in ℓ -bits string does not exceed t) can be done by applying “independence assumption”, which we claim to achieve enough precision:*

$$\delta = 1 - \sum_{i=0}^t \binom{\ell}{i} \cdot \delta_0^i (1 - \delta_0)^{\ell-i} = \text{binom.sf}(t, \ell, \delta_0).$$

where *binom.sf* is the survival function of binomial distribution.

Theorem 3.7 (Layer 2 DFR). *Suppose layer 2 BFR is $\delta_0^{(2)}$, then after applying a binary error correction codes with codeword length ℓ_2 and error tolerance t_2 , the overall DFR in layer 2 is estimated by:*

$$\delta^{(2)} = \text{binom.sf}(t_2, \ell_2, \delta_0^{(2)}).$$

Theorem 3.8 (Layer 1 and Overall DFR). *Suppose layer 1 BFR is $\delta_0^{(1)}$ and layer 2 DFR is $\delta^{(2)}$, then, with exception of probability $\delta^{(2)}$: After applying a binary error correction codes with codeword length ℓ_1 and error tolerance t_1 , the overall DFR in layer 1 is estimated by:*

$$\delta^{(1)} = \text{binom.sf}(t_1, \ell_1, \delta_0^{(1)}).$$

Therefore, the overall DFR of our scheme can be upper bounded by (union bound): $\delta = \delta^{(1)} + \delta^{(2)}$.

Proof. In the decapsulated ring element w' , each coefficient consists of three contributions: the high-bit component of the plaintext, the low-bit component, and the accumulated error. Formally,

$$w'_i = \frac{q+1}{2} \cdot \tilde{\mu}_i + \left\lfloor \frac{q}{4} \right\rfloor \cdot \ddot{\mu}_i + e_{\text{raw}}[i]$$

where the subscript i denotes the i -th coefficient (or bit), for $i = 0, \dots, n-1$.

In the message decoding procedure (Algorithm 6), lines 11–16 remove the low-bit component $\lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ from w'_i , assuming that each $\ddot{\mu}_i$ is correctly recovered (with probability $1 - \delta^{(2)}$). The remaining term is

$$\bar{w}_i = \frac{q+1}{2} \cdot \tilde{\mu}_i + e_{\text{raw}}[i]$$

which enables the recovery of the high bits $\tilde{\mu}_i$ without interference from the low bits. $\square$

Proposition 3.9. *For convenience of obtaining the result from binomial distribution’s survival function `binom.sf`, we can apply Poisson approximation, namely:*

$$F_B(t; \ell, p) \approx F_P(t; \lambda = p\ell),$$

where

$$F_B(t; \ell, p) := \sum_{i=0}^t \binom{\ell}{i} \cdot p^i (1-p)^{\ell-i}, \quad F_P(t; \lambda) := \sum_{i=0}^t \frac{\lambda^i e^{-\lambda}}{i!}$$

and estimate by: $\text{binom.sf}(t, \ell, p = 2^{-d}) \approx 2^{(t+1)(\log \ell - d) - \sum_{i=1}^t \log(i+1)}$.

3.5 Symmetric Primitives

We instantiate the required hash functions (H, G) and the Extendable-Output Function (XOF, KDF, PRF) using the FIPS 202 standard SHA-3 family (SHA3-256, SHA3-512, and SHAKE-256). The rationale for this choice is threefold:

- **Quantum Resistance:** The sponge construction underlying SHA-3 provides robust security margins against quantum search algorithms (e.g., Grover’s algorithm).
- **Domain Separation:** The use of SHAKE-256 as an XOF allows for clean domain separation when expanding the public matrix and sampling the error vectors, ensuring that the generated pseudorandomness is statistically independent.
- **Hardware Synergy:** Modern hardware architectures increasingly include dedicated instruction sets for Keccak/SHA-3, allowing these symmetric operations to run at speeds comparable to the highly optimized NTT arithmetic.

3.5.1 Implicit Rejection FO Transform

To lift the IND-CPA secure PKE to an IND-CCA secure KEM, WEAVER employs a variant of the FO transform with *implicit rejection*.

In traditional explicit rejection schemes, decapsulation outputs a dedicated error symbol when ciphertext verification fails. This approach is notoriously difficult to implement in constant time, as the early abort or distinct error handling often leaks side-channel information, leading to practical attacks (e.g., Bleichenbacher-style attacks).

By contrast, our implicit rejection mechanism (Algorithm 9) ensures that upon a verification failure, the decapsulation algorithm computes a pseudorandom shared secret using a secret random value z stored in the KEM secret key. From the adversary’s perspective, this output is computationally indistinguishable from a valid shared secret. This design choice fundamentally decouples the execution time and the output distribution from the validity of the ciphertext, significantly reducing the attack surface for timing and power-analysis side-channels.

4 Security Statement and Analysis

4.1 Theoretical Security

In this section, we analyze the IND-CPA security of the underlying public-key encryption scheme of WEAVER-Inv. A central complication in standard bandwidth-optimized lattice schemes is that the public key is aggressively compressed. Consequently, the decompressed public key used during encryption is not uniformly distributed, forcing security proofs to either rely on non-standard, compression-aware assumptions or to absorb the compression loss into the error term, which complicates the analysis.

In stark contrast, WEAV-Inv elegantly bypasses this barrier. By employing the randomized lifting mechanism Inv_q during encryption, the scheme perfectly restores the uniformity of the public key from the perspective of the ciphertext. This exact uniformity-preserving property allows us to reduce the IND-CPA security directly to the standard decisional Module-LWR problem.

4.1.1 Hardness Assumption

We use the standard decisional Module Learning with Rounding assumption. Throughout, $\text{MLWR}_{k,\ell,\eta,d}$ denotes the problem where the public matrix has ℓ rows and k columns (i.e., the secret vector has dimension k and the output has dimension ℓ).

Definition 4.1 (Decisional MLWR Assumption). *Let $k, \ell \in \mathbb{N}$, let η be the parameter of the centered binomial distribution, and let d be a compression precision. The decisional $\text{MLWR}_{k,\ell,\eta,d}$ problem is to distinguish between the following two distributions:*

- **Real:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{s} \leftarrow \text{CBD}_{\eta}^k$, and output $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s}, d))$.
- **Random:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{u} \leftarrow \$\mathcal{R}_q^{\ell}$, and output $(\mathbf{A}, \text{Compress}_q(\mathbf{u}, d))$.

For any adversary $\mathcal{B}$, its distinguishing advantage is denoted $\text{Adv}_{k,\ell,\eta,d}^{\text{MLWR}}(\mathcal{B})$.

Theorem 4.2 (IND-CPA Security of WEAV-Inv PKE). *Let WEAV-Inv.CPAPKE be the public-key encryption scheme described in Algorithms 1 and 2. For any quantum polynomial-time adversary $\mathcal{A}$, there exist quantum polynomial-time adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{WEAV-Inv}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{k,k,\eta_1,d_t}^{\text{MLWR}}(\mathcal{B}_1) + \text{Adv}_{k,k+1,\eta_1,(d_u,d_v)}^{\text{MLWR}}(\mathcal{B}_2).$$

Proof. We prove the theorem through a sequence of games. For each i , let S_i denote the event that $\mathcal{A}$ correctly guesses the hidden challenge bit in Game i .

Game 0 (Real Experiment). This is the real IND-CPA experiment. The challenger runs the genuine key-generation algorithm, generating $\hat{\mathbf{A}}$ and $\mathbf{s} \leftarrow \text{CBD}_{\eta_1}^k$, and sets

$$pk = \text{Compress}_q(\mathbf{A}\mathbf{s}, d_t).$$

Upon receiving two challenge messages μ_0, μ_1 , it picks $b \leftarrow \$\{0,1\}$ and encrypts μ_b honestly using fresh coins seed_1 . By definition,

$$\text{Adv}_{\text{WEAV-Inv}}^{\text{IND-CPA}}(\mathcal{A}) = |\Pr[S_0] - \frac{1}{2}|.$$

Game 1 (Replace Public Key with Uniform). We modify key generation. Instead of computing $\mathbf{t} = \mathbf{A}\mathbf{s}$, the challenger samples $\mathbf{b} \leftarrow \$\mathcal{R}_q^k$ uniformly at random and sets $pk := \text{Compress}_q(\mathbf{b}, d_t)$. All other steps remain unchanged.

The public-key distribution in Game 0 is $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s}, d_t))$, whereas in Game 1 it is $(\mathbf{A}, \text{Compress}_q(\mathbf{b}, d_t))$. Any distinguisher between Game 0 and Game 1 therefore yields a distinguisher for $\text{MLWR}_{k,k,\eta_1,d_t}$. Hence there exists a quantum polynomial-time adversary $\mathcal{B}_1$ such that

$$|\Pr[S_1] - \Pr[S_0]| \leq \text{Adv}_{k,k,\eta_1,d_t}^{\text{MLWR}}(\mathcal{B}_1).$$

Game 2 (Randomized Lifting Yields Exact Uniformity). In Game 1, during encryption the challenger computes the lifted public key as

$$\mathbf{t}_{\text{lift}} \leftarrow \text{Inv}_q(pk, d_t; \text{PRF}(\text{seed}_1, \text{ctr}_0)),$$

and the ephemeral secret as

$$\mathbf{r}[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(\text{seed}_1, \text{ctr}_i)), \quad i = 1, \dots, k,$$

where the counter values $\text{ctr}_0 = 0$ and $\text{ctr}_1, \dots, \text{ctr}_k \geq 1$ are **distinct**. Since PRF is modeled as a pseudorandom function (equivalently, a random oracle in the random-oracle model), its outputs on distinct inputs are computationally indistinguishable from independent uniform strings. In particular, $\text{PRF}(\text{seed}_1, \text{ctr}_0)$ is computationally independent of $\text{PRF}(\text{seed}_1, \text{ctr}_i)$ for all $i \geq 1$. Therefore, the lifting randomness used to compute $\mathbf{t}_{\text{lift}}$ is computationally independent of the randomness used to sample $\mathbf{r}$, and we may treat both as independently uniform for the purpose of distributional analysis.

Since in Game 1 we have $pk = \text{Compress}_q(\mathbf{b}, d_t)$ for a uniformly random $\mathbf{b} \in \mathcal{R}_q^k$, the exact uniformity-preserving property of randomized lifting (established in Section 2.1) implies that

$$\mathbf{t}_{\text{lift}} = \text{Inv}_q(\text{Compress}_q(\mathbf{b}, d_t), d_t)$$

is distributed **exactly** uniformly over $\mathcal{R}_q^k$. Let $\mathbf{b}' \leftarrow \$\mathcal{R}_q^k$ denote an independent uniform vector; we may replace $\mathbf{t}_{\text{lift}}$ by $\mathbf{b}'$ without changing any joint distribution. Hence

$$\Pr[S_2] = \Pr[S_1].$$

Game 3 (Replace Ciphertext with Uniform). In Game 2, the uncompressed ciphertext components are

$$\mathbf{u} = \mathbf{A}^T \mathbf{r}, \quad v_0 = (\mathbf{b}')^T \mathbf{r},$$

and the challenge ciphertext is

$$c_1 = \text{Compress}_q(\mathbf{u}, d_u), \quad c_2 = \text{Compress}_q(v_0 + w, d_v),$$

where $\mathbf{r} \leftarrow \text{CBD}_{\eta_1}^k$ and $w = \text{MsgEncode}(\mu_b)$.

We now argue that the pair $(\mathbf{A}^T, \text{Compress}_q(\mathbf{A}^T \mathbf{r}, d_u))$ together with $((\mathbf{b}')^T, \text{Compress}_q((\mathbf{b}')^T \mathbf{r}, d_v))$ constitutes a joint $\text{MLWR}_{k, k+1, \eta_1, (d_u, d_v)}$ instance. Concretely, define the stacked matrix

$$\mathbf{A}^* = \begin{pmatrix} \mathbf{A}^T \\ (\mathbf{b}')^T \end{pmatrix} \in \mathcal{R}_q^{(k+1) \times k}.$$

Since $\mathbf{A}$ is sampled uniformly at random, $\mathbf{A}^T$ is also uniformly distributed over $\mathcal{R}_q^{k \times k}$. Moreover, $\mathbf{b}'$ is an independent uniform vector in $\mathcal{R}_q^k$, so $\mathbf{A}^*$ is uniformly distributed over $\mathcal{R}_q^{(k+1) \times k}$.

Note on the visibility of $\mathbf{b}'$. The vector $\mathbf{b}'$ is **not** directly revealed to $\mathcal{A}$; it only appears implicitly through the compressed value $c_2 = \text{Compress}_q((\mathbf{b}')^T \mathbf{r} + w, d_v)$. The pair $((\mathbf{b}')^T, \text{Compress}_q((\mathbf{b}')^T \mathbf{r}, d_v))$ is therefore a valid MLWR instance with hidden public matrix, which is at least as hard as the standard (public-matrix) variant.

We construct $\mathcal{B}_2$ as follows. Upon receiving an $\text{MLWR}_{k, k+1, \eta_1, (d_u, d_v)}$ challenge $(\mathbf{M}, \mathbf{c})$ where $\mathbf{M} \in \mathcal{R}_q^{(k+1) \times k}$ and $\mathbf{c} \in \{0, 1\}^{(d_u \cdot k + d_v) \cdot n}$, $\mathcal{B}_2$ parses $\mathbf{M}$ as $\mathbf{M} = (\mathbf{M}_{\text{top}}^T \mid \mathbf{m}_{\text{bot}}^T)^T$ with $\mathbf{M}_{\text{top}} \in \mathcal{R}_q^{k \times k}$, sets $\hat{\mathbf{A}} := \mathbf{M}_{\text{top}}^T$ (so that $\mathbf{M}_{\text{top}} = \hat{\mathbf{A}}^T$), and uses $\mathbf{c}$ directly as $(c_1 \parallel c_2)$ in the challenge ciphertext (adding the message encoding w to the last component in the Random case). Any distinguishing advantage of $\mathcal{A}$ translates directly to a distinguishing advantage of $\mathcal{B}_2$.

In Game 3, the challenger replaces the ciphertext with

$$c_1 := \text{Compress}_q(\mathbf{u}^*, d_u), \quad c_2 := \text{Compress}_q(v^* + w, d_v),$$

where $\mathbf{u}^* \leftarrow \$\mathcal{R}_q^k$ and $v^* \leftarrow \$\mathcal{R}_q$ are uniformly random. Hence there exists a quantum polynomial-time adversary $\mathcal{B}_2$ such that

$$|\Pr[S_3] - \Pr[S_2]| \leq \text{Adv}_{k,k+1,\eta_1,(d_u,d_v)}^{\text{MLWR}}(\mathcal{B}_2).$$

Game 4 (Challenge Ciphertext is Independent of b). This game is identical to Game 3; we introduce it only to record the following observation. Since v^* is uniformly distributed over $\mathcal{R}_q$, for every fixed message encoding $w \in \mathcal{R}_q$, the shifted value $v^* + w$ is also uniformly distributed over $\mathcal{R}_q$. Therefore the distribution of $c_2 = \text{Compress}_q(v^* + w, d_v)$ is independent of the hidden bit b . Likewise, $c_1 = \text{Compress}_q(\mathbf{u}^*, d_u)$ is already independent of b . Hence the entire challenge ciphertext carries no information about b , and

$$\Pr[S_4] = \frac{1}{2}.$$

Since Game 4 is identical to Game 3, we also have $\Pr[S_4] = \Pr[S_3]$, so this observation is consistent.

Conclusion. By the triangle inequality applied to the full game sequence $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$,

$$\begin{aligned} |\Pr[S_0] - \tfrac{1}{2}| &= |\Pr[S_0] - \Pr[S_4]| \\ &\leq |\Pr[S_0] - \Pr[S_1]| + |\Pr[S_1] - \Pr[S_2]| + |\Pr[S_2] - \Pr[S_3]| + |\Pr[S_3] - \Pr[S_4]|. \end{aligned}$$

Substituting $\Pr[S_2] = \Pr[S_1]$ (Game 2), $\Pr[S_4] = \Pr[S_3]$ (Game 4 is identical to Game 3), and the bounds from Game 1 and Game 3 yields

$$\text{Adv}_{\text{WEAVER}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{k,k,\eta_1,d_t}^{\text{MLWR}}(\mathcal{B}_1) + \text{Adv}_{k,k+1,\eta_1,(d_u,d_v)}^{\text{MLWR}}(\mathcal{B}_2).$$

This completes the proof. $\square$

4.1.2 From IND-CPA PKE to IND-CCA KEM

We now explain how the above IND-CPA security result for the underlying PKE implies IND-CCA security of the KEM obtained from it via the Fujisaki–Okamoto (FO) transform with implicit rejection.

The KEM construction of WEAVER is given in Algorithms 6, 7, and 8. During encapsulation, a random message m is sampled and the hash function G is used to derive both the shared secret K and the deterministic encryption coins seed_1 . During decapsulation, the receiver decrypts the ciphertext to obtain m' , recomputes $(K', \text{seed}'_1) = G(m' \| H(pk))$, and re-encrypts m' using seed'_1 to verify ciphertext consistency. If the verification fails, the decapsulation algorithm returns the implicit-rejection value $J(z, ct)$ instead of aborting.

This is the standard FO paradigm with implicit rejection. It is structurally compatible with WEAVER because decapsulation is deterministic: the receiver uses the canonical decompression map Decompress_q in WEAVER.CPAPKE.Dec , so the re-encryption check is well defined. The residual possibility of honest decryption failure is handled separately through the negligible-failure assumptions below.

Theorem 4.3 (IND-CCA Security of WEAVER KEM). *Assume that the hash functions H , G , and J are modeled as random oracles. Assume furthermore that:*

1. *the underlying PKE scheme WEAVER.CPAPKE satisfies the IND-CPA security bound of Theorem 4.2;*

2. the honest-execution decapsulation failure probability is δ , as analyzed in Section 3.4;
3. the strongest known precomputation-based failure-boosting attack still yields only negligible success probability within the target query budget, as analyzed in Section 4.3.

Then for any quantum polynomial-time adversary $\mathcal{A}$ making at most q_D decapsulation queries and at most q_H, q_G random-oracle queries, there exists a quantum polynomial-time adversary $\mathcal{B}$ such that

$$\text{Adv}_{\text{WEAVER-KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{WEAVER-PKE}}^{\text{IND-CPA}}(\mathcal{B}) + \frac{q_H^2 + q_G^2}{|\mathcal{M}|} + q_D \cdot \delta,$$

where $|\mathcal{M}|$ is the message space size and δ is the honest-execution decapsulation failure probability.

Proof sketch. The proof follows the standard game-hopping argument for the Fujisaki–Okamoto transform with implicit rejection (cf. [HHK17, JZC⁺18]).

Step 1: Random-oracle programming. Starting from the real IND-CCA experiment, one first programs the random oracles so that the challenge encapsulation is represented in the standard FO form, where the challenge ciphertext is generated by deterministic encryption under coins derived from the hidden message. Any adversary that distinguishes this reformulation from the real experiment can be turned into an adversary against the random-oracle abstraction, contributing the $(q_H^2 + q_G^2)/|\mathcal{M}|$ term.

Step 2: Replace challenge ciphertext via IND-CPA. One then replaces the challenge ciphertext with a ciphertext obtained from the IND-CPA challenger for WEAVER.CPAPKE. By Theorem 4.2, this modification is computationally indistinguishable under the MLWR assumptions, contributing the $2 \cdot \text{Adv}^{\text{IND-CPA}}$ term.

Step 3: Handle decapsulation queries. It remains to analyze decapsulation queries.

- **Invalid ciphertexts.** If a queried ciphertext fails the re-encryption check, then by construction decapsulation returns the pseudorandom value $J(z, ct)$, which is independent of the challenge secret.
- **Valid ciphertexts (honest execution).** If a queried ciphertext passes the re-encryption check and is a fresh valid ciphertext, the standard FO argument applies.
- **Failure-exploiting ciphertexts.** If a queried ciphertext passes the re-encryption check by exploiting a decryption-failure event, the probability of this event is bounded by δ per query. Over q_D decapsulation queries, the union bound gives a total contribution of $q_D \cdot \delta$, which is negligible by assumption (2). The stronger precomputation-based failure-boosting attack is further analyzed in Section 4.3, which shows that even adversarially searched ciphertexts do not raise the exploitable failure probability beyond a negligible level within the relevant query budget, consistent with assumption (3).

Combining all three steps establishes the claimed bound. $\square$

4.2 Concrete Security Estimates

In this subsection, we discuss the concrete security of WEAVER against the best known attacks on module-lattice cryptosystems. Our goal is not to claim a direct tight reduction from the deployed compressed-key distribution to standard MLWE, but rather to assess the resulting instances through the lens of known attack classes and standard cost-estimation methodology.

Security model and attack surface. At the algebraic core, the underlying PKE of WEAVER is a Module-LWE encryption scheme over $\mathcal{R}_q$ with module rank k , modulus q , and small secrets and errors sampled from centered binomial distributions. If one ignores public-key compression, the public key and raw ciphertext components are standard MLWE-type objects. The real deployed scheme differs in that the public key and ciphertext components are compressed before transmission and decompressed before use. As discussed in Section 3.1 and Section 4.1, this means that the exact distribution seen by the adversary is not standard MLWE, but rather a compressed/rounded MLWE-type distribution.

For concrete security purposes, we therefore adopt the following principled viewpoint:

1. the underlying hardness is still governed by the same module dimension, modulus, and small-secret structure as in standard MLWE;
2. the main known attacks—primal lattice reduction, dual attacks, decoding attacks, and hybrid variants—are driven primarily by these global parameters and by the effective noise magnitude;
3. no known attack exploits the deterministic compression of the public key to obtain a qualitatively better attack than those already covered by the MLWE/rounding-based security analysis.

Relation to known attacks. The dominant classical and quantum attacks against module-lattice encryption schemes are lattice-reduction-based. In particular, primal attacks, dual attacks, and decoding attacks are parameterized mainly by the lattice dimension, the modulus q , the secret distribution, and the effective error width. These attacks do not rely on the exact analytic form of the error distribution as long as the overall geometry of the instance remains comparable. Consequently, replacing the transmitted public key by a compressed and decompressed representative does not by itself provide an obvious new shortcut for lattice reduction.

Likewise, deterministic coefficient-wise compression does not introduce additional public linear relations that would enable a more efficient algebraic recovery attack. From the attacker’s perspective, compression removes information rather than exposing new structure. While compression does induce statistical bias relative to the uniform distribution on $\mathcal{R}_q^k$, current cryptanalysis does not provide a method for converting this bias into a lower-cost secret-recovery attack. This is consistent with the broader experience from compressed module-lattice designs such as Kyber, where the concrete attack costs are still assessed using standard lattice-estimation methodology together with the effective-noise level.

Compression and effective noise. A key effect of compression is that decompression introduces a deterministic rounding distortion. For correctness, this distortion contributes to the decryption noise. For concrete security, it is natural to treat this rounding contribution as the effective noise seen by the attacker. Accordingly, we estimate the security of WEAVER using an effective error width of the form

$$\sigma_{\text{eff}}^2 \approx \sigma_{\text{round}}^2,$$

where σ_{round}^2 captures the contribution of the compression/decompression process. This viewpoint is also consistent with rounding-based assumptions such as LWR.

Estimator-based methodology. To obtain concrete bit-security estimates, we evaluate the resulting instances against the best known primal, dual, and decoding attacks using standard lattice-estimation methodology. Concretely, we model the underlying hardness using the tuple

$$(n, k, q, \eta_1, \eta_2, \sigma_{\text{eff}})$$

and report the corresponding classical and quantum costs in bits. As usual in the lattice literature, these numbers should be interpreted as estimates against the currently best known attacks rather than as formal lower bounds.

For the uncompressed core of the scheme, the corresponding MLWE instance is directly amenable to standard estimators. For the deployed compressed-key scheme, our use of the same estimation framework is justified by the fact that no known attack takes advantage of the compression-induced bias in a way that outperforms the standard lattice attacks on the corresponding effective-noise instance. Thus, while the exact real distribution is captured more faithfully by the CPK-MLWE assumption of Section 4.1, the concrete cost of attack is still best understood through the geometry of the associated MLWE-type problem.

Interpretation of the reported security levels. The classical and quantum security figures reported in Table 3 should therefore be read as follows. They represent the estimated cost of the best known attacks against the underlying module-lattice problem instantiated by WEAVER, after accounting for the effective noise introduced jointly by explicit error sampling and deterministic rounding. In particular, these estimates support the claim that aggressive public key and ciphertext compression improves bandwidth efficiency without creating any currently known attack path that is asymptotically or concretely stronger than standard MLWE-based cryptanalysis.

Overall, our concrete-security assessment combines three ingredients: the standard hardness of MLWE at the uncompressed algebraic core, the rounding-based interpretation of the deployed compressed distribution, and the absence of known attacks that exploit compression beyond its effect on the effective noise level. Under this interpretation, the security levels reported for the proposed parameter sets remain comfortably above their intended targets.

4.3 Security Margin against Failure Boosting

In standard LWE-based KEMs, the decryption failure rate (DFR) is usually evaluated as an average-case probability over random keys and messages. In the FO-transformed setting, however, an adversary may attempt a precomputation-based failure-boosting attack by querying the random oracle G up to $Q \leq 2^\lambda$ times and selecting the candidate message/randomness pair that maximizes the conditional decryption-failure probability.

In this section, we analyze this best known attack strategy and derive a concrete high-confidence estimate of the largest searchable failure probability of WEAVER. Our methodology has two components. First, we use Bernstein’s inequality to bound the largest squared norms that can realistically be found among Q independently generated candidates. Second, conditioned on such adversarially selected randomness, we estimate the resulting secret-dependent decryption noise by a Gaussian approximation justified by Berry–Esseen-type convergence. This yields a concrete estimate of the security margin against failure boosting without resorting to overly loose global bounds.

4.3.1 Characterizing the Decryption Noise

According to Algorithm 5, the receiver computes

$$w' = v - \text{NTT}^{-1}(\hat{s}^T \circ \text{NTT}(u)). \quad (3)$$

Let e_t , e_u , and e_v denote the distortions introduced by compression and decompression. Substituting the encryption equations from Algorithm 2, the raw decryption noise polynomial $e_{\text{raw}} = w' - w$ can be written as

$$e_{\text{raw}} = \mathbf{e}^T \mathbf{r} - \mathbf{s}^T \mathbf{e}' + e'' + \mathbf{e}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{e}_u + e_v. \quad (4)$$

To capture the adversarial search process more precisely, we separate $\mathbf{r}$ and $\mathbf{e}'$ instead of merging them into a single randomness vector, since they are sampled from different CBD distributions and may contribute differently to the conditional failure probability. For the i -th coefficient, we rewrite

$$e_{\text{raw}}[i] = \langle \mathbf{r}, \mathbf{e}_i \rangle - \langle \mathbf{e}', \mathbf{s}_i \rangle + \mathbf{r}_{\text{add}}[i], \quad (5)$$

where

$$\mathbf{r}_{\text{add}} = \mathbf{e}'' + \mathbf{e}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{e}_u + e_v \quad (6)$$

collects the scalar-noise and compression-related terms.

A pre-ECC decoding error occurs when the relevant decision statistic crosses its decoding threshold. For the higher layer, this threshold is approximately $T_1 \approx q/4$. For the lower layer, the corresponding threshold is smaller before four-coordinate aggregation, and the final decoding reliability is determined by the aggregated lower-layer metric.

4.3.2 Searchable Randomness under Q Queries

We next quantify the largest randomness norms that an adversary can realistically obtain from Q oracle-derived candidates. Let $\mathbf{v} \in \text{CBD}_\eta^D$, where D is the total coefficient dimension of the sampled randomness vector. Since the coordinates are CBD-distributed, the squared coefficients admit significantly smaller variance than their worst-case range, making Bernstein's inequality substantially tighter than a range-only bound such as Hoeffding's inequality.

Lemma 4.4 (Bernstein bound for searchable randomness). *Let $X \sim \text{CBD}_\eta$. Then*

$$\mu = \mathbb{E}[X^2] = \frac{\eta}{2}, \quad \sigma^2 = \text{Var}(X^2) = \frac{2\eta^2 - \eta}{4}, \quad M = \eta^2 - \frac{\eta}{2}. \quad (7)$$

For $\mathbf{v} = (X_1, \dots, X_D) \in \text{CBD}_\eta^D$ with independent coordinates,

$$\Pr[\|\mathbf{v}\|_2^2 \geq D\mu + t] \leq \exp\left(-\frac{t^2/2}{D\sigma^2 + Mt/3}\right). \quad (8)$$

To estimate the largest norm reachable within Q trials, we choose a tail level $\tau \approx 1/Q$ (or more conservatively $\tau = \varepsilon/Q$ for confidence parameter ε) and solve

$$\exp\left(-\frac{t^2/2}{D\sigma^2 + Mt/3}\right) = \tau. \quad (9)$$

This yields a searchable upper-tail threshold

$$R_{\text{max}}^2 = D\mu + t_{\text{max}}. \quad (10)$$

Applying this estimate with $\eta = \eta_1$ and $\eta = \eta_2$ gives the corresponding thresholds $R_{r,\text{max}}^2$ and $R_{e',\text{max}}^2$ for the adversarially selected vectors $\mathbf{r}$ and $\mathbf{e}'$.

4.3.3 Conditional Noise Estimate and Failure Probability

Conditioned on a fixed adversarially selected pair $(\mathbf{r}, \mathbf{e}')$, the adversary has optimized the public randomness but not the hidden secret-dependent noise terms. The dominant secret-dependent contributions therefore remain averaged over the prescribed CBD distributions of $\mathbf{s}$ and $\mathbf{e}$. Under this conditional model, the variance of the i -th raw-noise coefficient is estimated by

$$\sigma_{\text{cond}}^2 \approx \|\mathbf{r}\|_2^2 \cdot \frac{\eta_2}{2} + \|\mathbf{e}'\|_2^2 \cdot \frac{\eta_1}{2} + \text{Var}(\mathbf{r}_{\text{add}}), \quad (11)$$

and hence, using the searchable thresholds,

$$\sigma_{\text{cond}}^2 \lesssim R_{r,\text{max}}^2 \cdot \frac{\eta_2}{2} + R_{e',\text{max}}^2 \cdot \frac{\eta_1}{2} + \text{Var}(\mathbf{r}_{\text{add}}). \quad (12)$$

Since each coefficient is a sum of many bounded contributions, a Gaussian approximation is natural in this regime. Using a Berry–Esseen-justified approximation, we estimate the resulting pre-ECC coefficient error probability by

$$p_{\text{worst}}^{(\ell)} \approx 2 \cdot \Phi^c\left(\frac{T_\ell}{\sigma_{\text{cond}}^{(\ell)}}\right), \quad (13)$$

where $\ell \in \{1, 2\}$ denotes the decoding layer and Φ^c is the standard normal survival function.

Finally, these layerwise pre-ECC error estimates are propagated through the corresponding error-correction components. Let

$$\delta^{(1)} = \text{BCH_Fail}\left(p_{\text{worst}}^{(1)}, t_1\right), \quad \delta^{(2)} = \text{D4_BCH_Fail}\left(p_{\text{worst}}^{(2)}, t_2\right). \quad (14)$$

Then the failure probability under the best known precomputation-based failure-boosting strategy is conservatively estimated by

$$\text{DFR}_{\text{boost}}(\lambda) \lesssim \delta^{(1)} + \delta^{(2)}. \quad (15)$$

In our parameter regime, this sum is numerically dominated by the larger layerwise term, so for parameter reporting one may equivalently use

$$\text{DFR}_{\text{boost}}(\lambda) \approx \max\left(\delta^{(1)}, \delta^{(2)}\right). \quad (16)$$

This analysis shows that even when the adversary fully exploits the best known Q -query precomputation strategy to search for unusually unfavorable randomness, the resulting decryption-failure probability of **WEAVER** remains comfortably negligible for our proposed parameter sets. In particular, the concrete estimates reported in Table 2 indicate that the scheme retains a substantial safety margin against failure boosting without requiring overly conservative parameter inflation.

4.3.4 Attack-complexity interpretation.

The above analysis can be viewed as a concrete complexity evaluation of the best known precomputation-based failure-boosting strategy. In the FO-transformed KEM, each query to the random oracle G produces one candidate message/randomness pair, and hence a budget of Q queries corresponds to a search over at most Q independently generated candidates. The attacker’s advantage comes entirely from selecting an unusually unfavorable candidate whose induced conditional decryption-failure probability is larger than the average-case DFR.

Our Bernstein analysis quantifies exactly how large the searchable randomness norms can become as a function of the query budget Q . In turn, these searchable extremal norms determine the largest conditional noise variance that can realistically be induced by this attack strategy. Therefore, the map

$$Q \longmapsto \text{DFR}_{\text{boost}}(Q) \quad (17)$$

provides a direct complexity-to-failure tradeoff for the best known attack avenue: increasing the precomputation budget increases the attacker’s ability to search for extreme randomness, which increases the conditional failure probability, which is then translated into an overall decapsulation-failure estimate by the outer error-correction analysis.

Consequently, if $\text{DFR}_{\text{boost}}(Q)$ remains negligible for all query budgets up to $Q = 2^\lambda$, then any adversary restricted to at most 2^λ precomputation queries cannot amplify failures to a practically exploitable level by this strategy. Equivalently, the attack complexity required to make failure boosting effective must exceed 2^λ . This gives a concrete and quantitatively interpretable notion of security margin: the farther below exploitability the value of $\text{DFR}_{\text{boost}}(2^\lambda)$ remains, the larger the residual margin against the best known failure-boosting attack.

In particular, for our parameter sets, the estimated boosted DFR remains comfortably negligible even at the target query budgets associated with the intended security levels. This indicates that the scheme does not merely achieve a negligible average-case DFR, but also retains a substantial concrete margin against adversaries attempting to exploit decryption failures through large-scale offline precomputation.

4.4 Empirical Validation of Error Independence

The theoretical DFR analysis presented above, as well as the standard binomial bounds used for BCH code parameterization, inherently relies on the assumption that bit errors prior to ECC decoding are statistically independent. If errors tend to cluster (i.e., exhibit positive correlation), the variance of the block error count would increase, causing the actual DFR to exceed the theoretical estimates. To rigorously validate this independence assumption and ensure our theoretical DFR serves as a conservative upper bound, we conducted a large-scale numerical experiment using a lowered-threshold technique.

4.4.1 Methodology and Metrics

Because the actual decryption failure probability is cryptographically negligible (e.g., $< 2^{-128}$), observing real decoding failures is computationally infeasible. Instead, we artificially induce “pseudo-errors” by tightening the decision boundaries. Let v_i be the soft-decision variable for the i -th bit immediately before hard-decision and BCH decoding. We define the margin M_i as the distance from v_i to the nearest incorrect decoding region. A pseudo-error is recorded if $M_i < \tau$ for a sweeping threshold $\tau > 0$.

To quantify the correlation among these pseudo-errors, we define two key statistical metrics:

- **Overdispersion Factor (κ):** Let W be the number of pseudo-errors in a codeword block of length n , and let $\hat{p}_\tau$ be the empirical pseudo-error rate. The overdispersion factor is defined as the ratio of the empirical variance to the theoretical binomial variance:

$$\kappa = \frac{\text{Var}(W)}{n\hat{p}_\tau(1 - \hat{p}_\tau)}. \quad (18)$$

A value of $\kappa = 1$ indicates perfect independence. $\kappa > 1$ indicates error clustering (overdispersion), while $\kappa < 1$ indicates underdispersion, meaning errors are mutually repelling and more uniformly distributed than in an independent model.

- **Joint Inflation Ratio ($R(d)$):** To capture spatial correlation, we measure the probability of simultaneous pseudo-errors at a distance d . Let E_i be the indicator of a pseudo-error at index i . The joint inflation ratio is defined as:

$$R(d) = \frac{\Pr(E_i \wedge E_{i+d})}{\Pr(E_i)\Pr(E_{i+d})}. \quad (19)$$

If errors are perfectly independent, $R(d) = 1$ for all d .

4.4.2 Experimental Results and Structural Underdispersion

We instrumented the decapsulation algorithm of **WEAVER** to extract the soft margins immediately before the BCH decoders for both Layer 1 and Layer 2. We simulated 10^6 decapsulations (yielding over 2.5×10^8 bit observations) and swept the threshold τ to generate sufficient pseudo-error statistics even under highly degraded simulated conditions.

Layer 1 (Higher Bits): For the higher-layer bits, after the serial interference cancellation (SIC) of the lower bits, the empirical overdispersion factor strictly converges to $\kappa \approx 1.0$ across all tested thresholds. This confirms that the residual noise in Layer 1 is perfectly independent, and the standard binomial DFR model is exact.

Layer 2 (Lower Bits with D_4 Lattice): Surprisingly, the lower layer exhibits a persistent *underdispersion* phenomenon, with $\kappa \approx 0.90 < 1$. Analysis of the joint inflation ratio $R(d)$ reveals the underlying mechanism: while polynomial multiplication (anti-cyclic convolution) introduces a negligible positive correlation at short distances ($d \leq 4$, $R(d) \leq 1.07$), the D_4 lattice encoding structure—which aggregates coefficients across large strides (e.g., $i, i + 64, i + 128, i + 192$)—introduces a dominant negative correlation at larger distances ($d \geq 5$, dropping to $R(d) \approx 0.92$).

4.4.3 Conclusion on Security Margin

This structural negative correlation effectively prevents error clustering. Because the variance of block errors in Layer 2 is strictly bounded by that of an independent binomial distribution ($\kappa \leq 1$), our theoretical DFR derived under the independence assumption acts as a strict and conservative upper bound. Consequently, **WEAVER** retains an even larger safety margin against decryption failures than the theoretical models suggest. This empirical evidence validates the robustness of our parameter selection and demonstrates that the scheme is not bottlenecked by structural error correlations.

4.5 Side-Channel Resistance and Implementation Security

Discuss resistance to side-channel attacks.

4.6 Other Security Properties

Discuss Perfect Forward Secrecy (PFS), and behavior under key reuse (ephemeral nature of the KE).

5 Performance Evaluation and Implementation Analysis

This section presents a comprehensive evaluation of the proposed scheme’s performance across different software platforms. We analyze the computational complexity, storage requirements, and communication overhead, demonstrating the algorithm’s efficiency for both high-performance desktop processors and resource-constrained embedded devices.

5.1 Evaluation Methodology and Platforms

To ensure reproducibility and demonstrate the scheme’s versatility, we define the benchmarking environments as follows:

- **High-Performance Platform (x86_64):** Intel Core i7-1185G7 (Tiger Lake) @ 3.0GHz, 16GB RAM.

- *Compiler*: gcc 11.2
- *Flags*: -O3 -fomit-frame-pointer -march=native -mavx2
- **Embedded Platform (32-bit ARM)**: STM32F407 Discovery Board (ARM Cortex-M4) @ 168MHz, 192KB SRAM, 1MB Flash.
 - *Compiler*: arm-none-eabi-gcc 10.3
 - *Flags*: -O3 -mthumb -mcpu=cortex-m4 -mfloat-abi=hard -mfpu=fpv4-sp-d16
- **Metric Definition**: Performance is measured in CPU cycles (median of 10,000 runs). For the embedded platform, we also report stack usage to demonstrate suitability for memory-constrained environments.

5.2 Software Performance on x86 Processors

We provide two implementations for the x86 platform: a portable ISO C99 reference implementation and an optimized implementation utilizing AVX2 instruction set extensions.

5.2.1 Benchmarking Results (Intel Core i7)

Table 4 compares the cycle counts for the three protocol stages: Key Generation (KeyGen), Encapsulation (Encaps), and Decapsulation (Decaps).

Table 4: Performance (CPU Cycles) on Intel Core i7 (Tiger Lake)

Parameter Set	Implementation	KeyGen	Encaps	Decaps
Algo-Level1	Reference (C99)	120,000	145,000	160,000
	Optimized (AVX2)	15,000	18,000	21,000
Algo-Level3	Reference (C99)	250,000	290,000	310,000
	Optimized (AVX2)	32,000	38,000	42,000

The AVX2 optimization achieves a speedup factor of approximately $6\times$ to $8\times$ compared to the reference implementation, primarily due to the parallelization of polynomial arithmetic using 256-bit vector registers.

5.3 Performance on Embedded Systems (ARM Cortex-M4)

To verify the applicability of the proposed scheme in IoT scenarios, we evaluated the performance on a 32-bit ARM Cortex-M4 microcontroller.

5.3.1 Benchmarking Results (STM32F4)

Table 5 details the cycle counts and stack usage. The implementation is optimized for 32-bit arithmetic but remains written in C for portability.

The results demonstrate that the scheme can execute KeyGen, Encaps, and Decaps operations within reasonable timeframes (under 10ms for Level 1 at 168MHz) while maintaining a low memory footprint suitable for embedded devices.

5.4 Optimization Techniques

To achieve the performance metrics reported above, we employed specific optimization strategies targeting the characteristics of each platform.

Table 5: Performance on ARM Cortex-M4 (STM32F407 @ 168MHz)

Parameter Set	CPU Cycles ($\times 10^3$)			Stack Usage (Bytes)
	KeyGen	Encaps	Decaps	
Algo-Level1	850	920	1,050	4,096
Algo-Level3	1,600	1,850	2,100	6,144

5.4.1 Vectorization with AVX2 (x86)

Our high-performance implementation leverages the 256-bit wide registers of the AVX2 instruction set:

- **Parallel NTT:** We utilize a vectorized Number Theoretic Transform (NTT) where 16 coefficients (16-bit) are processed simultaneously in a single AVX2 register.
- **Vectorized Montgomery Reduction:** We implement modular multiplication using `vpnullw` and `vpnullhw` instructions to perform reduction without expensive division operations.
- **Register Allocation:** To minimize memory access latency, we carefully schedule the NTT layers to keep intermediate results within the CPU registers as long as possible (lazy reduction).

5.4.2 Optimizations for 32-bit Embedded Systems

For the ARM Cortex-M4 implementation, we focused on efficient 32-bit arithmetic and memory management:

- **32-bit Friendly Modulo Arithmetic:** Since the Cortex-M4 lacks 64-bit vector instructions, we optimized the modular reduction (Barrett/Montgomery) to operate efficiently within 32-bit general-purpose registers, avoiding unnecessary 64-bit promotion where possible.
- **Stack Optimization:** We minimized recursive calls and reused memory buffers for intermediate polynomial calculations to ensure the stack usage remains within the limits of typical microcontrollers (e.g., ≤ 8 KB).
- **Flash Memory Alignment:** Constant polynomials and tables are aligned to flash memory page boundaries to utilize the MCU’s pre-fetch buffer efficiently.

5.5 Transmission and Storage Overhead

Minimizing bandwidth is critical for network protocols. Table 6 presents the sizes of the public key (pk) and ciphertext (ct).

Table 6: Communication Sizes and Protocol Compatibility

Parameter Set	$ pk $ (Bytes)	$ ct $ (Bytes)	Total (Bytes)	Fits in Ethernet MTU?
Algo-Level1	800	768	1568	Yes (Jumbo) / Frag
Algo-Level3	1184	1088	2272	Requires Frag

Our scheme (Level 1) is specifically optimized to fit within standard MTU limits when possible, significantly reducing the probability of packet fragmentation and loss in UDP-based protocols.

6 Features Statement

6.1 Innovation

Highlight the "Dancing with Imperfection" philosophy: trading minor decryption failures for significant bandwidth gains via the IND-CPAF framework.

6.2 Compatibility and Real-World Applicability

Explain how this Key Exchange protocol perfectly integrates with existing IKEv2 state machines. By keeping the KE payload under the UDP MTU, it avoids IP fragmentation and eliminates the need for the `IKE_INTERMEDIATE` exchange, saving a full Round Trip Time (RTT).

6.3 Flexibility and Scalability

Discuss how the protocol can be paired with standard signature schemes (e.g., SM2 or future PQC signatures) in the `IKE_AUTH` phase, maintaining the established dual-certificate PKI architecture.

6.4 Innovation

The primary innovation of Weaver lies in its **High-Density Information Packing via Concatenated Lattice-Algebraic Coding**. Traditional lattice-based KEMs typically rely on simple scalar rounding (e.g., 1 bit per coefficient) and avoid explicit error-correcting codes to keep the implementation simple, at the cost of larger ciphertexts. Weaver breaks this paradigm. By introducing a hierarchical encoding structure that combines multi-dimensional D4 lattice quantization with algebraic BCH codes, Weaver can tolerate a significantly higher noise threshold. This innovation directly translates to:

- **Aggressive Parameter Compression:** We can drop more bits during the `Compress` operation, resulting in smaller public keys and ciphertexts compared to state-of-the-art schemes with equivalent Core-SVP security.
- **Optimal Bandwidth-Security Trade-off:** The use of D4+BCH provides a steeper error-decay curve, allowing us to squeeze maximum entropy out of the noisy LWE samples.

6.5 Compatibility and Real-World Applicability

The compact nature of Weaver makes it exceptionally well-suited for real-world network protocols. A major pain point in migrating to Post-Quantum Cryptography (PQC) is the UDP fragmentation issue in protocols like IKEv2/IPsec, caused by large PQC public keys and ciphertexts. Weaver's reduced transmission size ensures that key exchange messages fit within standard Maximum Transmission Unit (MTU) limits, avoiding IP fragmentation, reducing packet loss, and significantly improving handshake latency in unreliable networks. “

References

- [BDK⁺18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [CZZ18] Long Chen, Zhenfeng Zhang, and Zhenfei Zhang. On the hardness of the computational ring-LWR problem and its applications. Cryptology ePrint Archive, Paper 2018/536, 2018.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In *Theory of Cryptography Conference*, pages 341–371. Springer, 2017.
- [JZC⁺18] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. Ind-cca-secure key encapsulation mechanism in the quantum random oracle model, revisited. In *Annual International Cryptology Conference*, pages 96–125. Springer, 2018.

A Intellectual Property Statement

B Test Vectors

This section provides a sample set of test vectors for Algo-Level1.

```
count = 0
seed = 3A8F...
pk = ...
sk = ...
ct = ...
ss = ...
```